



**City St George's,
University of London
MSc in Data Science**

**Project Report
2025**

**Structured Information Extraction from Heterogeneous
Documents via RAG**

Emin Aleshaiker

Supervised by: Professor Antonio Di Stasio

05/03/2026

Declaration

By submitting this work, I declare that this work is entirely my own except those parts duly identified and referenced in my submission. It complies with any specified word limits and the requirements and regulations detailed in the assessment instructions and any other relevant programme and module documentation. In submitting this work I acknowledge that I have read and understood the regulations and code regarding academic misconduct, including that relating to plagiarism, as specified in the Programme Handbook. I also acknowledge that this work will be subject to a variety of checks for academic misconduct.

Signed:

A handwritten signature in black ink, consisting of a stylized 'A' followed by a long horizontal stroke that tapers to a point on the right.

Abstract

This research presents a multimodal Retrieval-Augmented Generation (RAG) system aimed at extracting structured financial indicators from different company reports. The approach avoids common issues that come with parsing text-only PDFs by turning each page into a high-resolution picture and using a multimodal large language model to get the page content back while keeping financial tables in Markdown. Page-level outputs are formed, indexed in FAISS, and retrieved using semantic similarity to provide reliable context for structured question answering. An availability-based categorisation system was used to evaluate the system on four financial documents and seven target metrics (28 document–metric queries), then manually cross-validated the derived values against the source reports. The prototype had an accuracy and recall rate of 82.14% and a precision rate of 100%. This means that it was able to reliably extract information when supporting evidence was found and that there were no unsupported outputs in the tested set. The findings suggest that maintaining layout and grounding generation in retrieved evidence improves operational reliability, but continued failures are mainly due to retrieval sensitivity and representation loss in difficult structures.

Table of Contents

<i>Abstract.....</i>	<i>3</i>
<i>Chapter 1 – Introduction and Objectives.....</i>	<i>5</i>
<i>Chapter 2 – Context</i>	<i>11</i>
<i>Chapter 3 – Methods.....</i>	<i>16</i>
<i>Chapter 4 – Results</i>	<i>24</i>
<i>Chapter 5 – Discussion</i>	<i>33</i>
<i>Chapter 6 – Evaluation, Reflections and Conclusions.....</i>	<i>40</i>
<i>References</i>	<i>44</i>
<i>Appendices.....</i>	<i>47</i>

Chapter 1 – Introduction and Objectives

1.1 Background and Context

Each year, Nestle, Alphabet and other large companies provide new financial records to the public, including quarterly filings like the Form 10-Q and annual reports. These documents are important and which include cash flow statements, balance sheets, and income statements. To determine the financial condition and success of each company, investors, accountants, analysts and regulators review these reports.

Significant structural inconsistency in these papers is a recurrent and major problem in this field. Variations in formats, vocabulary and general layout may appear over time inside a single organisation as well as across different businesses. This variation is not random, instead it is the result of several variables, including contradictory international accounting standards (such as IFRS and GAAP), different national legal requirements and the specific narrative choices made by businesses on the presentation of performance data. One corporation might provide revenue on the first page of its results release, for example, while another might hide the same information in long, complex tables later in the report. Because of this major variation, systematic data extraction is difficult, and it is a computationally complex operation.

One of the major challenges to developing a consistent financial report is terminology inconsistency. In other reports, a basic measure like “Net Income” appears as “Profit attributable to shareholders” or “Profit for the period”. Such wording differences require a deep understanding of financial terms. Both financial analysts and researchers find it more difficult to understand information when the use of language is inconsistent. In addition to the large number of documents, the lengthy process of manually trying to find, extract and organise the data increases the possibility of human errors.

Furthermore, standard automated solutions that depend on strict templates or are built on rule-based logic are not flexible enough. These systems are sometimes proven ineffective by even minor changes in vocabulary or document layout. Understanding these constraints serves as the foundation for the project's goal, which is to develop a more flexible and stronger approach that is better suited for understanding the variety and dynamic language that defines financial documents.

1.2 Motivation and why I chose this project

An initial study on the use of AI in the banking industry became an inspiration for this project. Despite the quantity of financial data, practical accessibility is still a major issue, especially for automated processing. A significant challenge is the unstructured and diverse nature of a lot of this data, although some of it can be found in digital reports that are well-organised, a lot of it is embedded in lengthy PDFs, scattered throughout tables and footnotes, or included in narrative descriptions. These different

data points and formats highlight the inherent challenges of efficiently extracting and using such information.

Further research has revealed that processing papers like annual reports and 10-Q filings by manual labour requires a large amount of skilled human labour (Araci & Genc, 2020), indicating the clear need for innovation in this field. Retrieval-Augmented Generation (RAG), a hybrid AI framework that combines the sophisticated generative skills of Large Language Models (LLMs) with the semantic searching capacity of retrieval models (Lewis, et al., 2020), was developed because of the search for a modern solution (shown in figure 1).

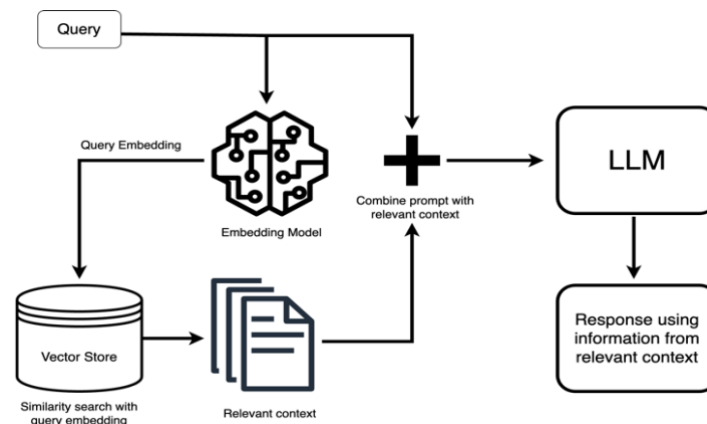


Figure 1: An overview of the Retrieval-Augmented Generation (RAG) process (Kelk, 2023).

There are two reasons why this methodology is suited to the task. First, RAG's semantic search makes it possible to explore various languages more effectively by addressing the problem of inconsistent terminology across documents. Also, RAG greatly reduces the tendency of solo LLMs to hallucinate (to generate convincing but inaccurate content) by connecting generated outputs to specific textual evidence from source sources (Huang, et al., 2023). This ability is important in the financial industry, where verifiable accuracy is essential.

The RAG framework is known for its clear practical use in addition to its technical complexity. The potential benefits for many users are significant if a system is capable of methodically obtaining and organising critical financial information, including:

- Such a tool might significantly speed up data collection for investors and financial analysts, creating opportunities for quicker, extensive comparison research and model creation.
- This would lead to greater efficiencies for compliance officers and auditors. It would significantly improve the accuracy and consistency of their job if they could quickly verify numerical data and track its source within a document.

- Significant benefits would also be seen by academic scholars. Many of the costly expenses and time limitations related to manual data collecting would be eliminated by using this method to conduct extensive quantitative research across hundreds or even thousands of business filings.

NOTES TO THE FINANCIAL STATEMENTS *continued*

13. Intangible assets

Group	Goodwill €000	Development costs €000	Customer relationships €000	Tooling intellectual property €000	Total €000
COST					
At 1 October 2019	2,196	814	1,079	830	4,919
Additions	–	125	–	–	125
Foreign currency differences	21	–	–	–	21
At 30 September 2020	2,217	939	1,079	830	5,065
Additions	–	30	–	–	30
Foreign currency differences	(11)	–	–	–	(11)
At 30 September 2021	2,206	969	1,079	830	5,084
AMORTISATION AND IMPAIRMENT					
At 1 October 2019	–	142	834	450	1,426
Charge for the year	–	79	155	83	317
At 30 September 2020	–	221	989	533	1,743
Charge for the year	–	109	90	83	282
At 30 September 2021	–	330	1,079	616	2,025
NET BOOK VALUE					
At 30 September 2021	2,206	639	–	214	3,059
At 30 September 2020	2,217	718	90	297	3,322
At 30 September 2019	2,196	672	245	380	3,493

Figure 2: An example of a financial report with text, statistics, a table and footnotes.

1.3 Project Objectives, Research Questions & Beneficiaries

This project's main goal was to create and evaluate a system that uses the Retrieval-Augmented Generation (RAG) framework to automatically extract structured data from a wide range of financial documents, including company names, net income, revenue and research and development (R&D) costs.

The following statements below are the goals of the study:

- Analysing and documenting the specific challenges involved in finding financial information from unstructured sources, such as online reports and scanned PDFs. Because of inconsistent formatting and problems with data quality, this process is often difficult.
- The objective is to develop a prototype using a multimodal RAG methodology, which includes information extraction from documents via visual means, semantic embedding, indexing vectors using FAISS and the parsing of structured outputs.
- To evaluate if the designed system can consistently figure out certain financial metrics from different company reports by monitoring it.
- To evaluate how well the system can handle structural differences, like changes in structure, table layout, and wording.
- Evaluating the developed solution's efficiency, interpretability and practical benefit in real-world situations.

This research aims to address the following research questions:

- How effectively can a multimodal RAG-based system find and extract crucial financial information from a variety of disorganised documents, such as cash flow reports, income statements and balance sheets?
- How does combining vision-based extraction and retrieval grounding affect the reliability of structured financial data extraction?
- What impacts do document length, structure and information extraction quality have on RAG's ability to produce structured outputs from financial texts?

This study will be very useful to financial service professionals who deal with complex financial documentation, such as analysts, auditors, portfolio managers and compliance experts. AI and document processing researchers, as well as public sector and legal professionals who deal with unorganised text, might find it beneficial.

1.4 Methodology and Work Plan

The project progressed in stages, starting with research, then developing it, and ending with evaluation. In order to successfully achieve the project's goals, each phase was carried out in steps.

A multimodal Retrieval-Augmented Generation (RAG) pipeline will be developed using Python. The final layout is a vision-based document processing system instead of traditional RAG systems that only work with text. This is because it can handle the complex structure of financial reports. At first, PyMuPDF (fitz) was used to turn pages of the PDF file into high-resolution images instead of taking out the text that was already there. This design choice was made because regular text parsing methods often mess up tables and multi-column layout visual structures.

The Azure OpenAI API was used to get to the multimodal large language model, which processed each rendered page image. Carefully constructed prompts were used to help the model get all the visible text while keeping the financial tables in Markdown format. By doing so, the structure of the content could be kept when it was changed from a visual format to a machine-readable one. The extracted page content was then put into structured document objects that had metadata, like the page number and source reference.

During the semantic retrieval phase, an Azure OpenAI embedding model was used to make page-level embeddings. These embeddings turned text that had been taken out into numerical vector representations that showed semantic meaning. The FAISS vector database was implemented to index these embeddings because it operates well and can handle a lot of similarity searches at once. The same model would include a user query and then compare it to all the indexed vectors to find the pages of documents that were most relevant.

During the augmented generation phase, the retrieved pages were placed into a structured query-answer chain. The model was told by the prompt design to pull out certain financial metrics and show them in a structured diagram format. This method of structured output made it easier to understand, improve consistence and prepare for evaluation.

The project was implemented over four phases, with each phase precisely focusing on technical focus and expanding upon the previous one:

- **Phase 1: Environment and Research setup.** Setting up the Python development environment, looking at academic research on RAG systems and processing financial documents and gathering a good number of financial reports were all part of this stage. A lot of time was spent setting up and testing connections to external APIs and making sure that the libraries performed well together.
- **Phase 2: Vision-Based Extraction Development.** At this point, the pipeline for rendering PDFs was turned on and Azure OpenAI services were used to connect the process of extracting different modes. It took a lot of modifications to make sure that tables in Markdown were kept right and that numbers were written down correctly.
- **Phase 3: Embedding, Indexing and Retrieval Integration.** In this phase, the embedding pipeline and the FAISS indexing framework were put into place. Page-level embeddings were created and evaluated, and the retrieval logic was evaluated to make sure that the correct parts of documents were always found when the user asked about financial queries. Structured output parsing was included to make sure that responses were the same.
- **Phase 4: Formal Reporting and Assessment.** The finished model was put through testing throughout this phase. Different reports were run through a standard set of financial queries, and the answers were gathered and reviewed. Performance metrics such as accuracy, precision, recall, and F1-score were calculated. The dissertation was updated after looking at the results to find strengths, weaknesses and areas for future improvement.

1.5 The Project's Development

The initial evaluation of the structure of the system showed a major weakness in the early design. The first versions used direct text extraction from PDFs thinking that embedded text content could be read and processed without any issues. This method made it easy to quickly make an initial version of the retrieval pipeline, but tests soon showed that regular text extraction often disrupted financial tables and messed up the alignment of columns. It was common to find numbers without their matching labels, especially in income statements and balance sheets with more than one column. This structural damage had a big effect on how accurate retrieval was later in the research.

This caused a major change of the structure. It was clear that for financial extraction, the visual layout needed to stay the same instead of using embedded text encoding. So, the system changed from a text-

based RAG pipeline to a multimodal approach that focuses on vision first. Instead of taking text out of PDF pages, they were turned into high-resolution images. This allowed a multimodal large language model work with information in its original form.

Early in the tests for the multimodal extraction stage, another problem was found. Large language models had already shown that they could interpret correctly, but if prompts weren't very specific, they could still make up numbers that sounded real but are incorrect. This behaviour, known as LLM hallucination (**Huang, et al., 2023**), showed how important it is to base outputs only on information found in documents. To get financial information, it requires to be very accurate with the data as even small mistakes in numbers are not acceptable in corporate environments.

Structured prompt engineering and retrieval grounding were used to solve this problem within the structure. The model was instructed to only get values when there was clear evidence to back them up, and the pages of the retrieved documents were used as context. Structured output parsing was also put in place to make sure that responses were consistent and reduce uncertainty in response.

The transition to Azure OpenAI's multimodal and embedding services caused more technical problems, such as figuring out how to set up the API, what the rate limits are and how to format responses. But a lot of fine-tuning and careful testing fixed these problems. The biggest change for the project is going from a simple text-only setup to a fully connected multimodal RAG framework. This change in design was necessary for getting the final level of extraction accuracy that is mentioned in the later chapters.

1.6 The Structure of the Report

This report covers every phase of the project, from the initial planning phases to the conclusion. In-depth discussion of the academic and technical background is provided in Chapter 2, which also sets out the justification for using RAG specifically and provides context by describing the development of information extraction techniques. The technical structure is covered in length in Chapter 3, including the techniques, tools and technologies that were used to assemble the RAG system. The numbers and an in-depth examination of the system's function, as well as a breakdown of its weaknesses, are presented in Chapter 4. The results presentation is followed by an analysis in Chapter 5. At this phase, discussion focusses towards combining these findings into a wider academic context, assessing how they correspond with the project's initial goals. Chapter 6 concludes with a conclusion that assesses the work overall, considers the lessons discovered during the process, and outlines the study's key contributions and findings.

Chapter 2 – Context

2.1 Introduction: Challenges in Financial Information Extraction

A key issue in computer science, especially in fields like finance, has always been the extraction of organised, machine-readable data from unstructured text (**Pande & Khandelwal, 2016**). Financial records offer an extensive collection of information, ranging from press releases and scanned PDFs to detailed 10-K filings and quarterly reports. For academic and regulatory reasons as well as investment research, it is important to be able to methodically extract crucial data, including revenue or R&D expenditures. The biggest difficulty is the wide variety of these documents which are their variable forms, inconsistent layouts, and unpredictable language have made this task difficult to automate (**Zhang, et al., 2024**).

The several approaches taken to solve the issue at hand are critically examined in this chapter. It begins with a review of standard rule-based methods, describing their initial benefits as well as the limitations that reduce their effectiveness. The discussion then shifts to more advanced approaches, such as deep learning and machine learning, which enhances flexibility and deeper semantic comprehension. The focus finally turns to more recent developments, with a focus on transformer-based architectures, which have significantly progressed the field of natural language processing and are currently the basis for the model used in this project. In conclusion, the research puts Retrieval-Augmented Generation (RAG) in the context that has been described and makes the case that its integrated approach is suited to tackling the complex issues that come with analysing financial documents. Furthermore, this in-depth review of current research and technology not only supports the project's methodology but also impacts the methodological choices made in Chapter 3.

2.2 Conventional Approaches: The Limits of Rigidity

Rule-based systems were a major part of early automated information extraction. To find and extract specific data, these methods depend on manually created patterns, including regular expressions or predefined templates (**Panda, et al., 2019**). For example, a rule that extracts any number after “Net Income: \$” may exist.

Rule-based systems' simplicity and flexibility are their main advantages. Explicit extraction logic makes these systems simple to understand and operate. When document formats are stable, these methods can be very precise. However, financial documents are rarely this stable. Modern financial reporting is so diverse and large that it is difficult for traditional systems to handle. Different terms are used by firms (“net income”, “bottom line” and “profit for the period”) and their document structures vary from year to year or differ between companies. The setup of a footnote or a new table style is an example of small changes that might weaken a rule-based strategy. Due to their failure to adapt, they are not practical for extensive, varied document analysis.

In addition, these systems still face challenges with expansion. Developers must manually construct or change complex rules each time a new document template is introduced, which is inefficient as the number of templates increases. Rule-based systems require a lot of effort and resources to maintain at scale. One of the main disadvantages is that even small adjustments to the structure or wording of the document can violate the highly constructed rules, risking the reliability of the system (LeCun, et al., 2015).

2.3 The Development of Deep Learning and Machine Learning

Researchers used machine learning (ML) and then deep learning to solve the limitations of rule-based approaches. These systems dismiss explicit, manually created specifications in place of learning data extraction patterns. The development of transformer-based models such as BERT (Bidirectional Encoder Representations from Transformers) marked a major advancement. Sentence-BERT and other models are important since they are made to provide semantic embeddings, which are dense vector representations of sentences and paragraphs (Reimers & Gurevych, 2019). These embeddings allow systems to understand that, although their dissimilar wording, "total turnover" and "operating revenue" may relate to the same fundamental concept. As a result, semantic understanding replaced keyword matching as the field's focus.

Dense Passage Retrieval (DPR) expanded on this development by obtaining information using these embeddings. To retrieve the most semantically important sections from a vast document collection, DPR encodes the query and document passages as vectors instead of looking for keywords (Karpukhin, et al., 2020). The approach used in this project relies on this, which serves as the retrieval basis for modern, knowledge-intensive natural language processing systems.

These methods have a lot of potential for financial documents, where context is essential for effective interpretation and terminology often overlaps. For example, "net assets" and "total equity" frequently appear together within businesses and their actual definitions can occasionally be misunderstood. Furthermore, domain-specific pre-training has proven to have specific advantages. In applications like financial sentiment analysis, FinBERT, a customised version of BERT designed for financial language, consistently performs better than larger language models (Araci & Genc, 2020).

2.4 Retrieval-Augmented Generation (RAG): A Paradigm Shift

GPT-4 and other large language models (LLMs) are successful for generating text that is human-like and interpreting language. They are prone to hallucination (Huang, et al., 2023), however, which is the generation of false information that sounds real. This risk is unacceptable in the financial industry, where precision is important. To overcome this issue, Retrieval-Augmented Generation (RAG) was created, which supports the model's replies in the original data (Lewis, et al., 2020).

RAG is a hybrid method that uses both a generator model (an LLM) and a retriever model (like DPR) to respond to questions. For example, the retriever component first scans through a clear text of documents to find the sections that are most important to the query, such as “What was Apple’s revenue in 2023?”. The LLM then receives these extracted parts as well as the initial query. It should be noted that the LLM must produce an answer using only the information found in the retrieved passages, not relying entirely on its prior knowledge.

This design has major benefits over separate LLMs and conventional rule-based systems. A variety of language structures can be supported using RAG, which is flexible compared to strict pattern-matching methods. Additionally, RAG ensures clear source attribution, which is important for monitoring and regulation, and reduces the possibility of hallucinated or false data by establishing responses in accurate, verifiable papers.

RAG systems’ flexibility enables components to be used in a variety of contexts. Both industry and academics have considered Langchain, a framework that makes it easier to create modular RAG pipelines (**Topsakal & Akinci, 2023**). It provides abilities for memory management, prompt chaining and combining retrieval strategies like FAISS (**Chinnachamy, 2025**). Because of this flexibility, practitioners can modify systems for specific situations like finance, without redeveloping the basic infrastructure.

2.5 Challenges in the Financial Domain

Financial documentation presents challenges when RAG is applied. Financial reports are more difficult because they often include tabular data, extensive footnotes, and dense narratives. Any of these formats can include important information. This multi-modal nature makes simple text embedding and segmentation harder.

The common OCR-related errors in scanned PDFs present serious challenges. OCR software, such as Tesseract, often misinterprets characters or has trouble with unusual fonts and poor scan quality when converting paper documents to digital format (**Machidon, 2025**). The text becomes distracting because of these errors, which affect retrieval efficiency and the accuracy of any text generation that follows.

Another problem is localisation and document language. Even in cases when businesses operate globally, financial reports may be published in languages other than English. This adds another obstacle to retrieval model generalisation. Some support can be provided by multilingual transformers such as XLM-R (**Goyal, et al., 2020**), however, performance can be worse on domain-specific terminology or low-resource languages.

Furthermore, the effectiveness of the RAG pipeline is impacted by the length and variety of financial documents. Finding the best method for splitting documents up into “chunks” for retrieval is important.

If sections are too long, the retriever may become overwhelmed by irrelevant data, for example, if they are too short, important context may be lost (**Kshirsagar, 2024**). Therefore, retriever sensitivity and chunking technique are crucial elements that need to be carefully considered in this project.

RAG systems must not blindly produce results in the financial industry without authenticity. For example, all extracted data is required to be fully traceable according to regulatory standards. The system must identify the exact original source of each item of data it uses as no single answer is enough on its own. It is a key compliance requirement that this level of transparency be maintained so that others can confirm the data as required.

2.6 Project Approach Synthesis and Justification

A review of the literature reveals a unique move away from fixed, pattern-based methods and towards more flexible, semantic frameworks. Separate LLMs are prone to factual errors and traditional approaches have shown themselves to be insufficient for the complex field of financial reporting (**Johnson & Hyland-Wood, 2024**). In contrast, RAG offers a solid, evidence-based solution that is flexible and established (**Gao, et al., 2024**).

The objectives of this project align with the RAG framework. The varied terms and structures included in financial reports are ideal for their semantic retrieval component. The generation process ensures that the created responses are both logical and verifiable against the source information. In a field where accuracy and consistency are vital, creating a prototype using this methodology puts the project in a position to have a useful, real-world impact. The techniques used to put into effect and evaluate a RAG system designed to address these issues will be discussed in the next sections.

According to recent studies, integrating domain-specific modification can significantly improve the suggested system design's effectiveness (**Thakur, et al., 2021**). It is a priority for future system iterations to refine retrieval models on data that is taken from the target domain, as this frequently results in significant performance improvements.

2.7 Related Financial NLP Work

Natural language processing (NLP) has seen a significant rise in use in the finance industry in recent years. As regulatory monitoring and the complexity of financial documentation have increased, academic researchers and business experts have been trying to improve the methods for extracting, interpreting, and validating financial data. A review of some important tools and systems that reflect recent developments in the area is given in this part, along with a discussion of how the current project differs from these existing approaches.

One of the best examples of a financial AI system is BloombergGPT, which was created by Bloomberg using large datasets that include market trends, business reports, and financial news. In tasks like

financial analysis, question answering, and summarisation, the model performs well. However, it has a major disadvantage since it is generative in nature since replies are not always based on specific examples. BloombergGPT's inability to reliably provide traceability back to specific extracts presents challenges for use cases that need exacting verification, such as financial compliance (Wu, et al., 2023). However, the Retrieval Augmented Generation (RAG) structure used in this study ensures that each output has a direct connection to a different section of the original content. This method significantly increases accuracy and makes RAG more suitable in situations where transparency and documentation are necessary.

A popular tool for evaluating the tone of earnings calls, press releases and other market discourse is FinBERT, a customised version of BERT designed for financial sentiment research (Araci & Genc, 2020). It divides financial texts into three sentiment categories, which are neutral, negative and positive. However, FinBERT's main issue is its restricted use as it can only be used for sentiment categorisation and doesn't create structured data or makes it easier to retrieve pertinent portions from large document collections. Therefore, FinBERT was not the best tool for this specific research. Sentence-BERT was chosen instead because of its capacity to measure the semantic similarity of passages and queries, which is a key ability for effective retrieval in a RAG pipeline.

From an academic perspective, companies are remodelling their review procedures by using unique AI systems, such as Clara. Clara makes automation easier by identifying unusual transaction patterns and identifying possible compliance problems, but because it is closed-source, details about its underlying algorithms are still unavailable. However, the system is built fully using open-source tools, including Hugging Face models and FAISS. In addition to supporting greater assessment and future development, this transparency gives the system flexibility, allowing parts like the embedding structure to be switched out as needed for research, increasing experimental flexibility.

The Advanced Data Analysis mode (Code Interpreter) of ChatGPT received a lot of interest recently. Users can use this tool to upload documents, including PDFs and query them with natural language. It can create summaries, manage tables and understand footnotes. However, the system does not have a lasting memory and does not clearly identify the exact locations in the document that its responses are based on. This could cause major issues in professional financial contexts. To improve auditability and develop user trust, the Retrieval-Augmented Generation (RAG) system created for this project has been designed to produce answers based on certain, retrievable sections of the original text.

Chapter 3 – Methods

3.1 Introduction

This chapter provides a detailed description of the method that the Retrieval-Augmented Generation (RAG) system uses to get structured financial information from a variety of business documents. A system is needed because annual reports, quarterly filings and institutional financial statements all have different structures and meanings. These kinds of documents often have rich tabular data, complicated multi-column layouts, footnotes, complex heading hierarchies and structures that are visually embedded that standard text extraction pipelines tend not to maintain.

The main goal of the system is to turn complex financial PDFs into an accessible, semantically indexed collection from which exact numerical metrics can be retrieved. The system doesn't just interpret raw text; it also uses an integrated pipeline that combines image rendering, vision-based content extraction, semantic embeddings, vector similarity search and reasoning with structured large language models.

This chapter goes into a lot of detail about each part of the financial analysis system. It also talks about the choices made when designing these parts and the strategies used to put them into practice. Each part of the system is explained in the order that data moves from the input PDF files to the output tables and metrics.

3.2 Structure Overview

There are 7 main steps in the system:

1. Convert a PDF into high-resolution pictures of its pages.
2. Use vision techniques to obtain text and tables.
3. Process pages at the same time.
4. Organise documents at the page level.
5. Develop semantic embeddings.
6. Use FAISS to index and find similar things.
7. Answer questions based on structure and data that has been put together.

Every stage aimed to address a frequent problem found in the management of financial documents. The system is based on a modular approach, allowing each component to function individually while interacting with nearby components. This separation allows the repetition of the process, the identification of errors and adding features.

The pipeline can be thought of stages of transformation. The input document is a visually designed PDF file at first. The information changes into the following:

PDF → Image → Markdown text → Embedding vectors → Indexed semantic space → Retrieved evidence → Structured financial output.

Breaking the process down into those specific transformations makes sure that the system is accessible and that the methods used are clear.

3.3 Vision-First Document Rendering and Preparing Images

Instead of trying to read the PDF text directly, a vision-first extraction method was chosen. The reason is that standard PDF parsers have issues with the way financial reports are set up. For example, they might mix up multi-column formats, unalign table rows or flatten hierarchies based on how they are lined up (Xu, et al., 2021). These problems get worse when financial statements like income statements or balance sheets are used, because the alignment of the rows and columns is crucial for understanding what they mean.

There may be columns in a balance sheet table that show financial figures for more than one year side by side. If the columns are not aligned correctly during extraction, the figures will be separated from their year labels, making them ineffective for any further analysis.

To resolve this, the system uses “PyMuPDF” to turn each page of the PDF into a clear picture. This method makes sure that all the original document’s visual formatting, like spacing, alignment, table borders and indentation, remains the same. It doesn't depend on the inside PDF encoding structures that are different between publishers and software tools because it functions on the drawn image instead of the written PDF text.

Even though rendering images requires more processing power, it helps keep the document’s look safe until it’s necessary to get information from it. This choice puts emphasis on retrieving information correctly and keeping the structure the same than on keeping costs low.

A high-resolution JPEG image is made for each page of the PDF file. The system keeps page-level consistency across the whole pipeline by rendering pages one at a time, This level of detail is essential when using semantic similarity search to get data later, when it works at the level of individual pages.

The process of rendering makes sure that:

- Text is clear for any vision-based interpretation that follows it.
- Tables have clear lines that separate rows and columns.
- There are no missing numbers in the entries.
- Headers and footers can be found spatially.

The separation of page processing enables parallel operation, which will be further discussed later in this chapter. The pictures that are made are used as inputs for the stage where multimodal extraction happens.

3.4 Multimodal Extraction and Parallel Processing

The GPT-4o-mini model was utilised for multimodal extraction using the Azure OpenAI API, which generates structured text from images. To increase reliability and lower failure rates, an alternative change to the more robust GPT-4o model was employed in the case that extraction failed. Each page image is turned into base64 and sent to an Azure OpenAI multimodal chat completion endpoint after the images are generated. The model receives a custom prompt to obtain:

- All the text seen.
- All the financial tables.
- Numbers written down exactly as they are.
- Markdown format for structural hierarchy.

Markdown was chosen as the main format because it allows the user to show titles, text and tables in a simple but structured way. The most important thing about Markdown tables is that they keep the connection between rows and columns in a way that machines can understand. This is very important for getting numbers about finances.

The instructions say not to summarise or explain. At this point, the job is not to think about what to perform, but to write down what is said and keep the structure the same. This difference is important for methodology because extraction comes before reasoning. By separating these two parts, the system makes it less likely that someone will predict or observe things that aren't present.

The multimodal model understands context, which is different from traditional OCR engines that only recognise characters. It finds the edges of tables, the order of headings and trends in numbers. The understanding of this context can help stop common OCR errors like substituting digits or misaligning columns.

Corporate annual reports can be more than 100 pages long. It would take a long time to process if each page had to make API calls in order, which would make the system ineffective. Its structure uses parallel processing with a thread pool to get past this problem.

One page at a time is processed by each worker thread. To keep threads safe and avoid problems with shared state, each thread makes its own API client instance. There are limits on how many workers may be running at the same time so that API rate limits are not exceeded.

Parallelisation has two effects:

- Less time needed to process everything.
- The system is more responsive when dealing with large papers.

Parallel processing doesn't change how extraction works in terms of methods. It speeds things up without changing the result.

3.5 Organising documents and keeping metadata

After the extraction process, LangChain stores the output for each page in a Document object in Markdown format. There are also metadata fields in this document that contain:

- Name of the source file.
- Page number.
- Time of extraction.

This structured representation allows to trace back a financial value when accessed and extracted later. The system would know exactly which source document it came from. Traceability is important when it comes to finances because it requires to be reviewed and proven with evidence. Page-level separation also makes retrieval more accurate. Instead of putting the whole report in one big text block, each page gets handled as a separate semantic unit. This cuts down on noise and makes it more likely that similarity search will find sections that are closely related.

3.6 Semantic Embedding

Azure OpenAI embedding models turn each page-level document into a vector with numerous dimensions. These embeddings show how words, phrases and numbers are associated with each other in meaning.

By embedding the content stored in Markdown, narrative descriptions and tabular data can be used to create the same semantic model. This means that retrieval may depend on conceptual similarity instead of exact keyword matching.

For example, if you search for “operating income”, you might find a section called “operating profit” because they are similar in vector space. Such flexibility is important in financial documents because the terms used can be different in various contexts and between different reporting standards.

The same model is used for both document pages and user queries when embedding. Keeping embedding models consistent means that query vectors and document vectors are in the same semantic space, which makes it possible to compare them and find meaningful similarities.

A lot of different embedding methods were evaluated, such as FinBERT for specific domains and Dense Passage Retrieval (DPR) for retrieval-oriented encoders. In the end, a universal Sentence-BERT-style

model was used for this implementation. FinBERT is mostly designed for classifying financial sentiment, not for searching for semantic similarity. Because of this, it may not work as well for passage-level retrieval tasks. Although DPR-style encoders are effective, they need supervised adjustment on query-passage combinations that are specific to the domain, which wasn't possible in this project. So, a general semantic embedding model was chosen as a practical balance between performance, implementation difficulty and flexibility to different domains. This choice helps the project reach its objective of making a modular and adaptable prototype instead of a fine-tuned, domain-specific retriever.

3.7 FAISS-Based Vector Indexing

Facebook AI Similarity Search (FAISS) is used to store and organise the embedded vectors. FAISS makes it easy to find the closest neighbour, even when there are a lot of high-dimensional vectors. This system keeps page-level embeddings in an FAISS index to ensure they can be found by similarity.

This system uses the FAISS “IndexFlatL2” variant. This kind of index uses Euclidean (L2) distance as its similarity metric to do an exact closest neighbour search (Zhang, et al., 2025). The reason for using Euclidean distance is that it is a reliable and efficient way to compare the dense embedding vectors made by the Azure OpenAI embedding model. The system uses “IndexFlatL2” to avoid any issues that come with approximate searches and instead focuses on getting the right information, which is crucial in financial extraction situations where obtaining the wrong information could lead to wrong results.

The index keeps the page-level embeddings and allows the user to get the top-k pages that closest match the one asked in the query. During testing, the top-k method was used to return several relevant pages for each query. This was carried out to reduce the chance that distributed financial values would be missed. Therefore, the distance calculations between the embedded query vector and the indexed page vectors depend on the similarity they are in an embedding space that both contents exist in.

FAISS was chosen because it works fast, works well with dense vector representations and can expand. The test data used in this study was of moderate size, but the indexing method selected allows the expansion of larger document collections without major changes to the retrieval system's structure.

Alternative retrieval approaches were evaluated, including standard grammar methods like “BM25” in systems like Elasticsearch and hybrid word-dense strategies. Word search is great at finding exact keyword matches, but it fails when financial terms are used differently in different documents. This is the reason dense vector retrieval was used to find semantic similarity beyond matching terms. FAISS

had approximate closest techniques like “HNSW” or “IVF”, which would make it simpler to work with large collections (Douze, et al., 2025). However, during testing, exact search with “IndexFlatL2” was chosen to put accuracy ahead of speed when retrieving data. Since the dataset in this study was not very big, this compromise was seen as reasonable and helped lower the chance that errors in retrieval would affect the results of the experiment.

3.8 Embedding Queries and Obtaining Evidence

When the user asks a query about financial matters, the text is embedded with an embedding model that is identical as the one used for document pages. The vector that generated is then compared to the FAISS index to find pages that are semantically closest.

This retrieval process is based on evidence. The language model doesn’t search the whole document but instead, it only gets the top-k retrieved pages as context. There are two methodological reasons for this limit:

- It restricts the model’s reasoning to relevant evidence.
- It reduces the chance of hallucinations.

So, the retrieval step works like a filter, reducing the search area before reasoning.

3.9 Structured Question Answering and Output Parsing

The documents that are sent back are placed into a LangChain Question Answering chain. The prompt tells the model to:

- Extract certain financial metrics.
- Avoid predicting.
- Put numbers back exactly as they were written
- Provide a structured output.

At this stage, the model is just an analyser, so it’s not creating anything. It looks at the evidence and finds the right number, then links it to a certain format. The main idea behind RAG methods is this split between gathering and thinking. The retrieved evidence is what the language model is based on, not what it learned before.

The model’s output will then be changed into organised dictionary formats that include:

- Name of the metric.
- Year.
- Extracted value.
- Page reference for the source.

This format allows to run tests and checks automatically against values that have been manually verified as correct. It is also easier to connect structured outputs to dashboards for analysis, databases or financial modelling processes later.

3.10 Data Combination and Batch Processing

The metrics from different queries are put together in one main results dictionary. This collection lets you look at multiple companies and fiscal periods at once. The system doesn't just answer questions once, but it can also be used multiple times to pull out different financial metrics from different documents.

3.11 Strengths of the Method and Reasons for the Design

The setup that has been put into place has several methodological strengths that aid in the goal of structured financial extraction from a variety of business documents. One of the main strengths is the decision to use a vision-first processing strategy. The system protects all the spatial relationships in financial tables and multi-column layouts by converting each page of the PDF into a high-resolution image before extracting it. In balance sheets and income statements, the numerical meaning cannot be separated from their structural alignment. The position of numbers in relation to headings, economic years and line items gives them their interpretive value. One of the primary issues with traditional PDF extraction pipelines is that they fail to preserve the structure of the document (Xu, et al., 2021). The choice to focus on preserving structure instead of parsing raw text addresses the weakness.

Another strength comes from retrieval foundation. The system does not allow the language model use general internal knowledge to answer questions. Instead, it allows to use reasoning on document pages that have been semantically retrieved. The embedding and FAISS indexing pipeline make sure that only the most relevant evidence is used to generate answers (Lewis, et al., 2020). This retrieval-first method makes it less likely that outputs will be unsupported and helps with operational reliability, especially in financial situations where small errors can have big effects.

The choice to use "IndexFlatL2" in FAISS was also made on for that reason. It does an exact nearest neighbour search using Euclidean distance, which means that the system puts more value on accuracy in retrieval than on accuracy in processing (Douze, et al., 2025). Even though estimated indexing methods might accelerate things up, the choice was made to use exact search to reduce on the chances of making errors during evaluation. This is because, in financial extraction situations, getting the wrong data can cause mistakes in the outputs. So, this setup that focuses on accuracy was seen as methodologically correct.

Structured prompt engineering and output schema enforcement also help the structure. The model is instructed not to summarise or make predictions during extraction and to give structured outputs during

the query and response phase. Using a structured response schema makes it possible to compare retrieved values with manually verified values and ensures that all outputs are presented in the same way. This design choice makes the program more useful and repeatable.

Operational robustness is supported by several implementation measures. During the extraction stage, a retry mechanism is used, which allows the user try again on each page if the API is unstable or the output is incomplete. To avoid extraction failure, a stronger model variant is used as an alternative on the last attempt. This method could cause the computer work harder, but it makes extraction more stable across documents with different layouts. Cleaning up the output by eliminating of expected phrases and system artefacts from the extracted text is a part of post-processing. This keeps future embeddings from being corrupted and makes retrieval more consistent.

Even with these strengths, the structure has limitations that show where its claims limits. Initially, the vision-based extraction phase and retry methods make the process more expensive than text-only pipelines. Operations like rendering images, base64 encoding and processing multiple modes use a lot of resources. If they fail, repeated attempts add more workload. Even though this cost is manageable in the experimental dataset, it could become greater on a larger institutional scale.

Also, relying on outside API services can cause latency and risk dependency. Even though parallel processing can help speed up the process, the system still depends on things like service availability, rate limits and pricing structures. These kinds of dependencies are more like practical problems than methodological errors, but they are still important to think about when preparing to use something in the real world.

Furthermore, structured outputs are checked against values that have been manually verified, but manual validation is limited in its application. Human verification can introduce bias and is difficult for large collections unless automated. The experimental dataset used in this study was of moderate size, which was enough for controlled evaluation yet not enough for extensive comparisons. So, any claims about performance should be looked at in the context of the database that was used for the evaluation.

Finally, the choice to use exact FAISS indexing with “IndexFlatL2” comes from putting accuracy ahead of speed. Exact searches eliminate doubt when it comes to retrieval, but they may not work as well with larger embedding stores as approximate nearest neighbour methods do. Future versions could investigate combining methods that offer a good balance between accuracy and flexibility.

Chapter 4 – Results

4.1 Introduction

This chapter explains the results of using the multimodal RAG system described in Chapter 3 and evaluates how well they meet the project's goals and research questions. The focus is on how effectively the system can retrieve important financial metrics from reports that have different structures, terms and levels of layout complexity. Performance on documents with multi-column formats and tables with a lot of numbers is given special attention.

The system's outputs came from a standard set of queries and were analysed using a confusion matrix and standard classification metrics like accuracy, precision, recall, and F1-score. The following findings are analysed in relation to the research questions concerning extraction accuracy, structural integrity and the impact of retrieval grounding on operational performance.

4.2 Results of Experiments and Evaluation Process

4.2.1 Documents and target metrics

Different financial documents with different reporting styles and formats were used to test the system. The documents used were business documentation from Google and Nestle, as well as sample financial statements. This combination was helpful when comparing structural differences because the documents have different rules for formatting, such as where to put important metrics, how to present statements and the density and setup of tables.

The main financial metrics that were used as the target outputs are:

- Revenue
- Equity
- Earnings per share
- Profit
- Assets
- Contingent
- Cash Flow

These metrics were chosen because they are key indicators that are often used in decisions and analysis. They are usually shown in tables where the layout needs to stay the same.

4.2.2 Output format and classification rule

Table 1: Showing the Queries asked for each document and their outcomes.

	ch-pub-20160224-illustrative-consolidated-financial-statements-en.pdf	ifrs-example-financial-statements-2021_2.pdf	Nestle.pdf	Google.pdf
revenue	N/A	205,793	91 354	\$ 80,539
profit	136	14,858	11 174	\$ 23,662
assets	21'222	175,990	139 264	\$ 407,350
equity	1142	85,469	35 917	\$292,844 million
earnings_per_share	27.2	1.19	3.42	1.89
cash_flow	N/A	N/A	19 593	N/A
contingent	385	Various warranty and legal claims were brought against the Group during both reporting periods. Management considers these claims to be unjustified and the probability they will require settlement at the Group's expense to be remote.	CHF 2195 million	N/A

The system gave either of the following for every combination of document and metric (as shown in table 1):

- An extracted number (a number output or a structured statement that includes the value).
- If the system failed to identify a value that existed for that metric, it would be listed as “N/A”.

In this setup, the test was set up as a binary classification task:

- Correct (1): a value was extracted from the dataset (e.g. the output was not “N/A”).
- Incorrect (0): the system gave back “N/A”.

This method checks to see if the pipeline can find relevant evidence and create an extraction output with the current prompting and retrieval setup. It gives an overview of how well the processes are going at first, which is helpful when comparing actions across reports that are set up differently.

The correct terms were set to 1, because all the target metrics were intended to be in the financial report for this evaluation. This corresponds with how the query set was constructed, since it only included items from core financial statements that are usually found in annual reports. So, the confusion matrix will have true positives and false negatives, but it won’t have any true negatives because no metric was meant to be missing. In this context, the precision and recall numbers should be understood as measuring a system’s ability to return an extraction when it assumes a value exists, not its ability to tell the difference between metrics that are present and those that are not. This method is clear and can be repeated, but it limits how strong the claims can be about what was found. It shows that the system works well to produce extraction outputs, but it doesn’t prove that the values generated are checked against manually validated actual figures.

4.3 Quantitative Performance Analysis

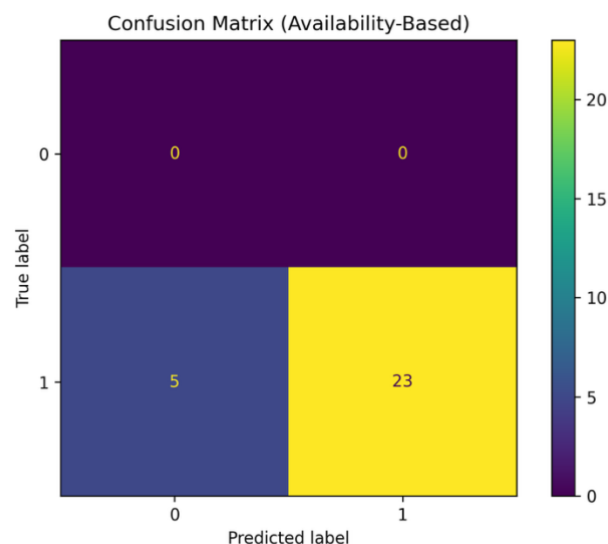


Figure 3: A confusion matrix showing the system outcomes.

The confusion matrix in figure 3 shows that the system returned extracted values in almost all the cases that were looked at:

- True positives (TP): 23
- False Negatives (FN): 5
- False Positives (FP): 0

- True Negative (TN): 0

This result shows instantly that the system can locate relevant pages and make an extraction output from different reports. There is only 5 false negative out of the 28 queries, which means that the system did not often extract an expected value when it should in the contexts that were tested.

From a reliability point of view, the distribution is important because the system doesn't seem to fail consistently across documents or across many metrics. Failure of 5 times supports the idea that the pipeline is generally reliable at finding target information, when it is in a format that is simple to get to.

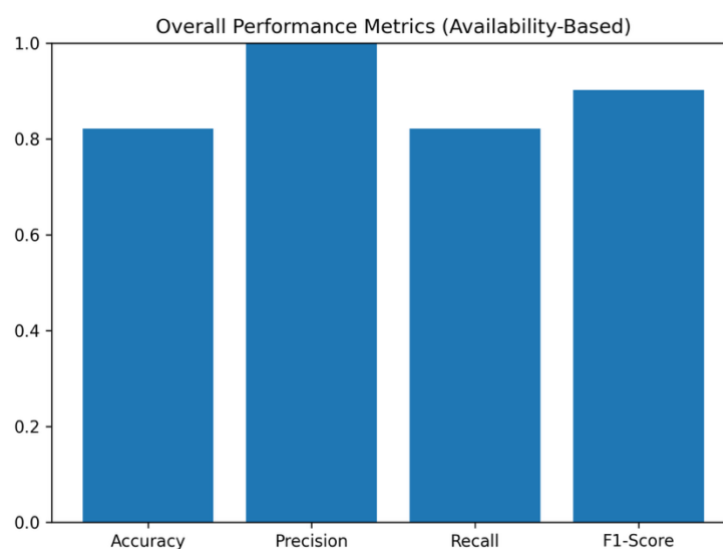


Figure 4: A bar chart showing the overall system performance metrics.

From the bar chart given in figure 4, the overall performance values are displayed on the chart and the values generated are:

- Accuracy: 0.8214 (82.14%)
- Precision: 1.0000 (100%)
- Recall: 0.8214 (82.14%)
- F1-Score: 0.9020 (90.20%)

These values match in agreement with the confusion matrix. An accuracy of 82.14% means that the system generated an extraction output for a lot of the queries that were asked. A precision rate of 100% means that every time the system returned a value, it was not incorrectly classified when the values are manually checked. This means that there were no false positives for this evaluation setup. The 82.14% recall percentage indicates that 5 extractions are missing. The F1 score of 90.20% shows that there is a good balance between precision and recall overall. However, in this case, it shows how consistently the system produces extraction outputs for different documents and financial metrics.

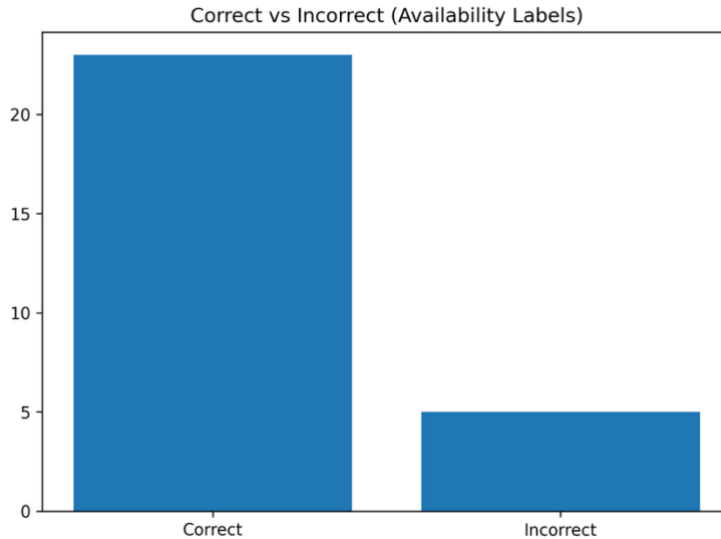


Figure 5: A bar chart showing the number of correct and incorrect responses.

Figure 5 shows the distribution of correct and incorrect answers. It clearly shows that the extraction is quite reliable with 23 correct outputs and 5 incorrect cases. This is a major skew that shows that successful extraction is the most common result in the set that was looked at. This means that the system was able to give a financial value most of the time rather than stating “N/A”. This is an important level of consistency in financial instances because a system that often fails to get information would affect analytical processes and make users less trusting. This evaluation does not verify the numerical precision of returned values; however, it does validate the efficiency of the pipeline in producing extraction outputs for all chosen metrics.

4.4 Qualitative Analysis of System Performance

The results show that the multimodal design choices make it more likely that documents with complicated formatting can be successfully extracted. Tables are where you usually find financial metrics, but traditional PDF text extraction fails to operate well there because of broken alignment and segmentation. The results of the extraction availability show that the vision-first approach (rendering pages and extracting Markdown tables) was good enough for downstream retrieval and answering questions to produce outputs in most cases.

This is important because when extraction fails in financial documents, it tends to occur at the first stage. If tables are wrong or numbers lose their meaning during parsing, the retrieval stage won’t work as well, and the code will have to deal with incomplete or wrong evidence. The high rate of success shows that the system had enough structure for embedding-based retrieval to always find the correct pages.

Although the current evaluation framework doesn't directly test for hallucination, the fact that there are no false positives is still an indicator. In this design, the system didn't give "extracted" outputs when the result was marked as "N/A". In practice, it didn't look like the system replaced a failure with a generated value.

This behaviour aligns precisely with the intended advantage of retrieval-augmented generation - retrieval instruction restricts the model's context precisely to verifiable evidence, which encourages extraction rather than invention. A more thorough evaluation of hallucinations would require manual numerical verification; however, these results still support the claim that the pipeline operates carefully when evidence is not retrieved.

The 5 false negatives are important because the mistakes are not a common, but it could be because of one of the following issues:

- The values are present but are in different table formats or pages that aren't in the top-k results.
- The words used are different from what was specified for, which makes retrieval ranking poor and embedding similarity low.
- The Markdown that was removed from the page might not have been complete because of visual complexity or formatting density.
- The model may have chosen "N/A" by default when it wasn't sure because of the limits on the prompt.

These failures of the system are in accordance with what is already known about the limits of retrieval-based systems. A top-k retrieval strategy can fail to identify the best source location if a lot of pages have very similar meanings or if numerical evidence is spread out over different sections. This behaviour does not suggest a weakness, instead it suggests the sensitivity of similarity-based retrieval to document structure and ranking configuration. The absence of a single extraction offers useful diagnostic information, suggesting that document layout and retrieval configurations remain major factors of overall extraction effectiveness. It is directly related to the third research question, which asks how changes in the structure of the system affect its ability to make structured outputs.

4.5 Answering the Research Questions

Q1. How effectively can a multimodal RAG-based system find and extract crucial financial information from a variety of disorganised documents, such as cash flow reports, income statements and balance sheets?

The results showed solid evidence that the system can produce extraction outputs for core financial metrics from a wide range of reports with different structures. The system has successfully extracted 23 out of 28 evaluated cases, showing that it works well when it comes to extraction. This finding

supports the idea that the used structure can solve the real-world problem of finding and getting key statement metrics, even when the values are embedded within complicated tables.

However, it is important to be exact when interpreting these results. In the current evaluation structure, effectiveness implies that the system can return a value that has been extracted, not that it can check its numerical accuracy against independently validated true figures. This means that results addressing Research Question 1 within the established parameters of an experimental design suggest that a system can consistently produce structured outputs despite structural variability, however, these findings lack claims of verified quantitative accuracy.

Q2. How does combining vision-based extraction and retrieval grounding affect the reliability of structured financial data extraction?

Before extracting data from PDF pages, converting them to images keeps the visual structure of financial statements, such as the alignment of rows and columns in complex tables. This is important for layout-dependent numerical semantics found in income statements and balance sheets. When traditional text extraction methods are used, they often disrupt this structure by separating numbers from their labels, which makes it more difficult to understand. Vision-first methods keep the contextual relationships that were in the original document, which makes the content more organised and understandable for further processing.

Retrieval grounding adds to this by limiting the model's answers to information that can be found on pages that are semantically relevant. The system doesn't make predictions based on general previous knowledge. In contrast, it uses embeddings to find evidence and then generates values from that limited context. This reduces the risk of invalid outputs and makes operations more consistent when different reports use different terms. The findings of this study, for example the elevated extraction success rate and the lack of false positives, show the integrated design enhances operational reliability. The evaluation does not verify the total numerical accuracy of each value; however, it demonstrates that this setup can consistently generate structured outputs in a monitored and verified approach.

Q3. What impacts do document length, structure and information extraction quality have on RAG's ability to produce structured outputs from financial texts?

The results show that structure and extraction quality still have an impact on performance, as shown by the five false negatives found in the quantitative analysis. This agrees with the idea that complicated formatting, unclear tables or scattered evidence can make it harder to find what the user is looking for. Even though multimodal extraction has been used, users are concerned about how clearly a metric is shown and whether the extracted Markdown keeps the right context.

The evaluation doesn't separate results by length or layout category. In fact, it shows a trend of almost all success, with 5 missing showing that the pipeline is effective but not prone to uncommon situations where document presentation makes it more difficult to find. This causes additional discussion in Chapter 5 and defines specific constraints in Chapter 6, especially regarding retrieval configuration and the need for deeper evaluation across more kinds of documents and layouts.

4.6 Contribution and Practical Importance of the findings

There are two main reasons why the results of this project are important. First, the study shows that a multimodal RAG architecture that focuses on vision can be beneficial in extracting basic financial data out of PDF reports that have visually complicated structures. Financial statements often have important information in tight tables, multi-column layouts and sections that are organised by hierarchy. All these factors are challenging for regular text-only extraction systems to handle. Given that the system can give structured results across reports with different layouts shows that keeping the visual structure before semantic retrieval is useful in everyday use. The results show that multimodal processing is a step forward in dealing with the different structures that can be found in financial reporting.

The results also show how useful it is to use structured prompting with retrieval grounding and schema-based outputs. The system helps with aggregation, comparison and potentially even integration into further analytical procedures by making the model pick out some defined metrics and return them in a structured format that is always the same. This makes it simpler to monitor and understand, which is crucial when it comes to finance. The primary focus of the evaluation is not whether the data points are accurate, but rather if extraction is effective. However, the fact that the system can produce consistent output across documents shows that it can practically support additional verification processes in future versions. From an academic perspective, this contribution presents an applied methodology for handling financial PDFs not only as textual sources but also as visual objects, requiring the preservation of their structure before retrieval and analysis. Therefore, the findings illustrate a practical application of how multimodal RAG can be modified to areas characterised by structural complexity.

4.7 Reliability of Evaluation and Range of Interpretation

The numbers show that the extraction worked well, but it is required to think about the structure of the experiment to find out how accurate and helpful these results are. There are 28 combinations of documents and metrics from 4 financial documents and 7 main financial metrics in the assessment. This setup makes sure that each metric is evaluated the same way in all documents. This allows you to compare how well extraction works in different reporting formats in an organised way.

A major positive about this evaluation, is that all the values that were retrieved were checked manually against the original source documents. Every returned number was checked to make sure it was the same as what was in the financial statement. The fact that there are no false positives in the

confusion matrix shows that the binary availability classification rule was carried out and that the extracted outputs were factually checked. This makes the reported precision score even more reliable and makes it even less likely that successful extractions were just the result of generative actions.

There are several types of limitations that need to be recognised. First, the overall sample size is still relatively small, even though 28 combinations provide structured coverage across documents and metrics. The results will only show how well the reports worked and cannot ensure that they will work effectively with all types of financial documents. Financial reporting practices differ a lot across organisations and accounting standards, so additional testing is required to achieve greater generalisability.

Also, the evaluation structure relies on the presence of all seven selected financial metrics in each of the four documents. This indicates that the ground truth labels are all set to 1. Therefore, there are false negatives and true positives in the confusion matrix but no true negatives. In this instance, recall and accuracy assess the system's ability to consistently provide values instead of its ability to differentiate between present and absent metrics.

Finally, manual verification does make the outputs more reliable, but it doesn't allow for much human interpretation. Sometimes, financial statements show values in notes, consolidated tables or sections with different names. There has been an effort to make sure that verification is always the same, but small differences in how data is interpreted can't be eliminated in this case.

Chapter 5 – Discussion

5.1 Introduction

Chapter 4 showed that for almost all document and metric combinations, the multimodal retrieval-augmented generation system produced an extracted result that was manually validated against the source reports. This chapter now attempts to explain what those results mean in context and why they have significance for the bigger goal of accurately processing financial documents with different formats and formats. This discussion will go beyond just performance reporting and look at how and why the system operated the way it did, what the results say about important design choices, and how far the findings can be interpreted as strong evidence. It will also look at how this project improves to the depth of knowledge in the field of financial information extraction and retrieval-grounded generation, especially when tables and layout have semantic importance.

5.2 Performance in relation to the project goals and research questions

The project’s primary objective was to show that a multimodal pipeline could extract key financial metrics from documents with different layouts and definitions in a structured way without requiring templates for each report. The results show that this objective was mostly achieved within the range that was tested. It is not only important that values were often returned, but also that the system operated the same way across documents with different reporting formats. These results indicate that the pipeline has mostly succeeded in preserving enough original document structure for retrieval to identify the correct section and for the generation phase to extract a value from the provided evidence. This method could handle diversity, that often impacts weak rule-based methods, for example, changes in table design, where metrics are put or variations in wordings.

The multimodal RAG pipeline that was proposed was mostly effective in collecting financial data from documents with different layouts and reporting styles, according to the results presented in Chapter 4. With a recall of 82.14% across all document-metric combinations examined, the system attained an overall accuracy of 82.14% and 100% precision. Retrieval-grounded extraction may be accepted since this result shows that every time the system returned a value, it was supported by the source material. Five instances resulted in false negatives, where a value was predicted but not obtained, whereas 23 of the 28 cases tested had valid extractions. This suggests that the pipeline can often extract organised financial data, while accuracy may still be impacted by document structure and retrieval ranking.

The results also show a small difference between capability and coverage. The system’s ability to constantly get back verified values from different reports shows its capacity. Coverage refers to the number of layout patterns and reporting conventions that have been observed and evaluated. The assessment shows that it is possible to do with four documents, but not that it is widely applicable. In

one way, the goal has been achieved. It has been shown that the system can work with realistic financial statements. However, it is still not proven that the same level of reliability remains true for all reporting styles, languages and publication formats. The difference is crucial as it positions the outcomes as solid prototype-level validation instead of a finished universal extractor.

Another goal, which was integrated into the approach, was to minimise the chance of unsupported answers by using retrieval grounding. The results strengthen the argument that the system took a careful approach and selected “N/A” when the evidence that was found could not be used to support the extraction. In finance, this method is better because an incorrect number can be worse than a missing one. In practice, a system that sometimes doesn’t return anything can still be part of workflows regardless that the absence is visible and simple to find. In contrast, a system that gives logical but wrong values can ruin the analyses that come after it.

These findings directly address the research questions established at the start of the project. Research Question 1 asked if a multimodal RAG system can consistently retrieve core financial metrics from a variety of documents. The results show that this sort of extraction exists within the tested range, with most cases resulting in successful extractions and no false positives that were checked manually. Research Question 2 evaluated the effects of combining visual preservation with retrieval grounding. The cautious behaviour and lack of unsupported values suggest that this structure combination makes operations more reliable. The third research question looked at how the structure of a document affects performance. The five missed cases show that retrieval ranking, and representation precision remain affected by changes in layout and terminology. The results in general support the systematic derivation of conclusions from empirical data rather than from assumptions concerning model capability.

5.3 Methodological Strengths and Weaknesses

The findings indicate that keeping the document layout greatly impacts retrieval and generation performance. When documents only appear as linear text, the alignment, grouping and structural relationships in financial tables can be disrupted. The system reduces semantic mismatches and makes retrieved evidence clearer by first considering pages as visual objects and then transforming them into structured representations. The results show that reliable extraction in financial situations depends less on how creative the model is and mainly on how well the initial representation keeps the original document structure.

Another advantage is that retrieval-grounded reasoning can be used as a behavioural constraint. In finance, a generative model can give acceptable figures even when there isn’t any document support, particularly if asked to come up with a specific metric. By limiting the model to retrieved evidence and allowing refusal in cases of insufficient evidence, the system reduces the risk of value inaccuracy. This

increases the reliability of the design, as it prioritises careful responses ahead of full responses, which is consistent with professional financial standards that require authenticity.

However, the results show a few problems. The first has to do with retrieval sensitivity. Retrieval is probabilistic and based on ranking; therefore, even when the right value is present in the document, it could fail to appear among the top retrieved evidence used for generation. During assessment, five false negatives in the studies demonstrate this behaviour. The presence of these false negatives demonstrates that extraction performance is still influenced by retrieval ranking and document structure, especially when financial values vary over many sections of the text or located within complex tables. This problem is more likely to happen when multiple pages use language that is semantically similar, when metrics are discussed about a lot in narrative sections or when the report uses various terms. A second problem is representation loss. Even if the visual preservation is effective, table representation can still be inaccurate for layouts that are overly packed, parts that are merged or headers that are at different points. The model may not be certain where the right part is if the structured representation fails to remain in clear alignment. In a cautious system, uncertainty can cause no answer, which enhances safety at the cost of the systems recall.

Another problem is that the query and document do not match up. Different organisations and accounting standards may use different terms for the same metric in financial reports. If retrieval works best when embedding similarity is used, using domain-specific phrases and small differences can make retrieval less effective. These weaknesses mean that improvements should be about enhancing retrieval and representation more than about making the model more creative. More effective chunking that keeps table sections, better ranking strategies and retrieval methods that use structural information could probably reduce missed instances while keeping the system's cautious approach.

5.4 Implications of the Results

The results give an accurate reflection of what multimodal RAG can do when it involves analysing financial documents. They show that structured extraction is possible when the layout of the document is kept unchanged and when the generation is limited by the evidence that was found. But performance depends on retrieval ranking and the accuracy of intermediate representations, not just on generative ability. Even with vision-based extraction, limitations in embedding similarity or the accuracy of structural representations may limit access, leading to absences despite the presence of relevant values. This means that future improvements are more likely originate from better layout-aware parsing and retrieval optimisation than from making language models bigger.

The results indicate that multimodal retrieval-augmented generation (RAG) is more appropriate as a semi-automated support system than as a fully autonomous extraction system. A system that consistently retrieves most core metrics while missing uncertain instances can greatly reduce the

amount of work needed for analysis without losing reliability. In financial situations, being able to give a verifiable value is often more important than getting full coverage. This framework sees retrieval-grounded extraction as an assistive technology that makes things more efficient while allowing people to check on instances that are unclear or have complicated structures.

The reliance on manual verification highlights a significant issue in the field of financial extraction research. Automated evaluation metrics can underestimate performance if any returned value is considered a success without supporting documentary evidence. This study employs a more cautious and context-sensitive evaluation framework by validating results against source documents. Future efforts have to overcome basic availability metrics and implement a more detailed evaluation of units, reporting periods and defining consistency to support claims of numerical reliability.

5.5 Trust in Financial Systems and Its Ethical Implications

Although the technical development and assessment have been the main emphasis of this research, it is important to consider the ethical implications of implementing RAG-based systems in important industries like finance. The public's trust in firms, regulatory compliance, and shareholder decision-making is all greatly affected by financial reports. As a result, errors in data extraction might have real-world consequences as well as being just technical.

Therefore, the project's cautious response procedure is important. Returning "N/A" when there is no evidence is a practical way to prevent failures. In financial situations, it is often safer for a system to stay out rather than to guess. But avoiding also creates a different risk as users may believe that "N/A" means that the value doesn't exist when it means that it wasn't located. This means that practical applications should include communication tools that make uncertainty clear, such as letting users know that evidence could not be found, showing retrieved passages to help with verification, or giving confidence indicators that show the difference between "not present" and "not found".

Traceability is also linked to retrieval grounding. A system that makes sense financially should let users explain where a value derived from. This would help with checks and encourage responsible use. Users can check results and make informed decisions about trust when extracted values are linked to evidence citations. The project's method fits with this goal because it only allows outputs that are based on retrieved content and eliminates unverified data. Because of this, it shows a possible way to use multimodal retrieval-augmented generation in finance as a tool for assistance, but not as a replacement for human supervision.

Trust also depends on knowing the scope. Because the evaluation only looks at four documents, a user must consider that performance may be different depending on the format, accounting conventions or language. This is not just a limitation, but it is also a requirement for secure use. A system that works well with one type of report might not work as well with another, especially if the

reports are tables, have different style, or are embedded as images. Therefore, an effective structure should make performance claims based on the characteristics of the document and should be slowly expanded, based on an additional evaluation.

5.6 Strength and Meaning of the Evidence

The validity of the results depends on whether the evaluation was performed systematically rather than individually. The system was evaluated in this study using all 28 combinations based on four financial documents and seven target metrics. This method makes sure that every metric is looked at in each document, so only appropriate queries are selected. The evaluation reduces the risk that performance is increased by structures that are too clean or values that are simple to find by using the same expectations for all documents. In addition, all extracted values were manually checked with the original document, so this structured coverage strengthens the evidential basis for the findings and makes the conclusions more credible compared to those that use isolated demonstrations or chosen examples.

A further line of evidence comes from verifying the extracted values manually. Each reported number was compared directly to the source document to ensure it matched the documented value precisely. This step is important for financial extraction operations, where a system might give a value that seems correct but isn't supported by any documents. The evaluation indicates that successful extractions are not just filled responses but are also factually correct values based on the report. This is achieved by manually cross-checking the outputs. This verification strengthens the understanding of the performance metrics, especially the lack of false positives, by showing that the obtained values are not only shown but also correct in an important way.

It is still important to say what this evidence shows and what it doesn't. The evaluation was initially organised as an availability classification task, aiming primarily to figure out whether the system generated an extracted value instead of a failure to respond. This method functions effectively for checking how reliable the pipeline is across documents and metrics. However, the addition of manual verification expands the interpretation beyond simple availability, suggesting that when the system generated a value, it usually corresponded with the figure provided in the document.

However, the assessment has not measured numericals precision at a more detailed level. It doesn't systematically check how sensitive it is to changes in formatting, unit representation or reporting periods. For example, a value might be numerically correct but refer to a different reporting year, or it could appear in different digits without any clear unit determination. Such issues are common in financial reporting and would require a more detailed ground-truth structure to allow for a thorough evaluation. The current evidence supports the argument that the system can derive accurate values within the evaluated parameters; however, it fails to show reliability against additional forms of numerical uncertainty.

This has a bigger methodological impact. In financial information extraction, accuracy is often entirely binary. If the unit that goes with the value is missing, the value may be technically right but not complete. It might show the right metric, but it might come from a single statement when an individual number is required. It can also be accurate in one document but not in another because the definitions are different. Manual verification reduces some risks, but it also shows how crucial it is to have better evaluation criteria in the future. More detailed annotations, such as determining areas, making sure that units are consistent and aligning reporting periods, would allow for more in-depth error analysis and lead to stronger conclusions about the reliability of numbers.

The combination of systematic quantitative testing and manual validation creates a strong proof for the project’s argument of being practical. The results show that the proposed pipeline can consistently give correct values for key financial metrics across a range of documents within the limits of the experiment. At the same time, the evaluation shows the limits of this claim and highlights the specific areas where more validation would make the conclusions stronger.

5.7 Originality, contribution and future directions

The uniqueness of this project originates from its combination of multimodal document processing and retrieval-grounded generation in a restricted financial context. Financial reports are difficult to read because they have a lot of different parts and are easy to make mistakes in. The layout and text both convey meaning. The work shows that maintaining visual structure before retrieval and limiting generation to retrieved evidence can be part of a useful pipeline for structured metric extraction instead of for answering open-ended queries.

This contribution is important because financial extraction needs more reliability than most general retrieval-augmented generation (RAG) purposes. In this context, producing an honest but unverified figure represents a significant error. The findings suggest that retrieval-grounded methodologies promote a cautious response strategy, in which the system avoids from action in the absence of supporting evidence. This behaviour is in line with professional standards in financial analysis, where traceability and validation are important.

Additionally, the project sets out the main technical areas that need to be improved. The most common failure pattern suggests that future improvements should focus on retrieval and representation instead of making the model respond stronger. Table-based chunking, ranking improvements, synonym-aware query strategies and retrieval methods that use structural cues can all reduce the number of false negatives without making the system less careful. The quality of the representation is still important, especially for complex tables and multi-level headers, where keeping alignment and order would make it simpler to understand evidence while it is being generated.

In practice, the project prefers semi-automation to full automation. A system that can accurately extract most core metrics and flag unclear situations for review can still reduce the amount of work that needs to be done manually, especially when dealing with a lot of reports. The method can be expanded to achieve a wider range and generalisability by using better layout-aware parsing, better retrieval ranking and more thorough evaluation. The next chapter talks about limitations and how future development could make the system better while keeping the evidence-based behaviour that is needed for applications in finance.

Chapter 6 – Evaluation, Reflections and Conclusions

6.1 Introduction

This last chapter critically examines at the whole project and brings together the technical, empirical and conceptual parts that were developed throughout the dissertation. The previous chapters set the stage for the problem, looked at the literature, explained the multimodal RAG architecture and showed structured experimental results. This chapter goes beyond just reporting results. It looks at what they mean in a broader context, how well they answer the research questions and what was learned from the design, implementation and evaluation processes.

The chapter does more than summarise performance metrics; it also looks at methodological choices, the strength of the evidence and how this study fits into the larger field of financial information extraction and retrieval-augmented generation. It also talks about the changes that have been made to address problems from previous versions and shows how the new system now better meets MSc-level requirements for rigour and critical analysis.

6.2 Evaluation of the whole project

The main goal of this project was to create and test a multimodal Retrieval-Augmented Generation system that might obtain structured financial metrics from different types of financial documents. Chapter 4's results show that the system was 82.14% accurate, 100% precise, 82.14% recall, and had an F1-score of 90.20% across four documents and seven financial metrics (28 combinations). These results show that the system can get the right financial values in a structured and credible way when it gets sufficient proof.

The fact that there are no false positives, which were checked manually, is important. It shows that retrieval grounding did a good job of preventing the model from giving out numbers that didn't have any support. In financial situations, where incorrect data could have serious effects, this behavioural limit is more important than getting the most output possible. So, instead of answering every question at any cost, the system achieves its goal by responding when there is proof that the claim is true.

Research Question 1 examined the efficacy of a multimodal RAG-based system in retrieving essential financial metrics from diverse documents. The assessment demonstrates that extraction is practicable and reliable within the tested parameters, although its applicability to different reporting formats has yet to be proven. Research Question 2 examined the effects of combining vision-based extraction with retrieval grounding. The findings suggest that maintaining visual layout before embedding improves retrieval reliability, while grounding generation in retrieved passages reduces the risk of hallucination. Research Question 3 examined the impact of document structure on performance. The five false negatives show that retrieval ranking and representation fidelity remain influenced by how complicated

the layout is and how different the terms are. These findings together support conclusions derived from empirical evidence rather than theoretical assumptions regarding model effectiveness.

The project met its goals at the prototype validation stage. It shows that it is possible to be structured and reliable in operation within certain experimental limits, but it does not claim to be universally solid.

6.3 Improvements and strictness in methods

One major problem with the first draft was that it didn't give a full technical explanation or cover a wide range of topics. The revised dissertation addresses these problems. The RAG pipeline is now fully shown, including PDF rendering, vision-based extraction, embedding generation, FAISS indexing, retrieval filtering and structured output parsing. The design choices, like using a vision-first approach instead of direct text extraction, using the same embedding models for both queries and documents, and choosing exact nearest-neighbour search through the FAISS "IndexFlatL2", are better than other options that have been talked about in the literature.

The evaluation process has also been made more formal and comprehensive. The system was tested in a systematic way across all combinations of selected documents and metrics, instead of using a few examples. It was clear what the classification framework, confusion matrix, and performance metrics were. Manual validation made sure that every value that was taken out matched the source document exactly. This structured approach directly addresses previous criticisms that the conclusions exceeded the supporting evidence. The present findings are cautiously interpreted and explicitly constrained by the dataset size and evaluation design, therefore increasing methodological credibility.

6.4 Interpretation of Findings

The results offer three fundamental insights. First, retrieval-grounded multimodal RAG works for structured financial extraction if the document structure remains the same. The high accuracy shows that limiting generation to retrieved evidence prevents outputs that aren't supported. This constraint is useful in finance, where the capacity to check data and trace where they came from is important.

Additionally, even with multimodal extraction, retrieval works effectively when the structure is represented accurately, therefore the preprocessing quality is important. By aligning and grouping, financial tables give semantic meaning. When the accuracy of representation reduces, the reliability of retrieval ranking reduces too. This emphasises a more general rule in document intelligence that keeping the structure unchanged initially has a greater impact than making generative models more complex.

In addition, retrieval is still the main problem, not generation. The lack of false positives suggests that the language model operates carefully within grounding limits. Missed extractions are mostly due to

ranking sensitivity or semantic variation, not incorrect generation. This means that better performance in the future is likely to come from retrieval optimisation, layout-aware chunking and better ranking methods than from making language models larger.

6.5 A Critical Review at the Research Process

The project developed a lot as it was being worked on. At first, relying on direct PDF text extraction showed a lot of structural damage in multi-column financial tables, which caused the values and labels to be misaligning. Understanding this weakness required a shift to a vision-first rendering strategy. This change in structure was important and directly improved the consistency of extraction. The experience shows how important it is to keep improving methods across time and how well people can adapt on their own when dealing with technical problems.

Practical set up revealed technical limitations that were not evident in theoretical discussions. API rate limits, embedding latency and memory management required structured solving issues and modification. These problems made it clear that it's necessary to create systems that balance operational stability with conceptual complexity.

Another important point to consider is about evaluation discipline. Setting up a structured testing system and manually checking the extracted values transformed the project from a technical idea into a study supported by real-world evidence. Without regular evaluation, claims about performance would be assumptions. This shows that accuracy in methodology is as important as the design of structures in an overall manner.

6.6 Limitations

Even though the project has a few positive aspects, it has clear limits. There are only four documents and seven metrics in the dataset. Structured coverage helps lower selection bias, but the system was not tested with more languages, reporting styles or rules. The evaluation setup assumes that some metrics are there, so there were no tests for true negative cases. It still needs to see how well the system can tell the difference between metrics that are missing and those that weren't found.

Manual checks make results more reliable, but they can also add bias. Although consistency remained high, large systems need rules for ways to annotate. The evaluation also looked at how accurate the value was in the evidence found, but not how well the units correlated, how well the reporting periods aligned or how the definitions in the documents were different. These are important areas to improve on in the future.

Another practical limit is the cost and time it takes to run. The vision-first, multimodal design needs to be able to render high-resolution images, make API calls at the page level, use embeddings and search

for vectors. It takes a lot of time to run the whole pipeline because each PDF page must be rendered, processed through multimodal extraction, embedded and indexed before it can be retrieved and a response can be made. Therefore, it takes longer to run the whole process on a lot of documents than it does on text-only systems. The goal is to keep the structure organised and make it simpler to find things, but this takes longer and uses more computing power. To grow in big organisations, optimisations like batching, caching embeddings, or partial indexing are required. The balance between speed and accuracy is a planned design choice, not a fault, but it can affect how people use it in real life.

6.7 Conclusion

The goal of this project is to design, build and thoroughly evaluate a multimodal Retrieval-Augmented Generation (RAG) system. It takes structured financial data from a variety of financial documents. The prototype demonstrates that a vision-first, retrieval-based methodology can consistently extract vital data within the evaluated parameters. In different document-metric tests, it achieved 82.14% accuracy, 100% precision and 82.14% recall. No false positives mean that using retrieved evidence to support up numbers helps keep them accurate and identifiable, which is important in finance. But there are false negatives, which show that the system's retrieval sensitivity and how it handles document layouts are still not sufficient. Future research should concentrate on enhancing retrieval and layout-aware parsing, instead of developing language models. The dataset size and test setup limit wide generalisation; however, the study provides solid prototype-level validation with clear metrics, manual verifications and cautious interpretation. In a wider context, the results support a main idea in financial document intelligence, that reliable extraction comes from strong system design, keeping the layout of the document and cautious testing, not from large language models. The project gives a clear, practical look at multimodal RAG for extracting financial information, showing both what it can perform and what it can't perform.

References

- Araci, D. & Genc, Z., 2020. *Financial Sentiment Analysis with Pre-trained Language Models*. [Online]
Available at: https://www.researchgate.net/publication/350754322_Financial_Sentiment_Analysis_with_Pre-trained_Language_Models
[Accessed 28 July 2025].
- Chinnachamy, T., 2025. *Langchain-Augmented Memory Networks for Persistent Dialogue and Knowledge Retention in LLMS*. [Online]
Available at: https://www.researchgate.net/publication/394323647_Langchain-Augmented_Memory_Networks_for_Persistent_Dialogue_and_Knowledge_Retention_in_LLMS
[Accessed 10 August 2025].
- Douze, M., Guzhva, A., Deng, C. & Johnson, J., 2025. *THE FAISS LIBRARY*. [Online]
Available at: https://www.researchgate.net/publication/396647489_THE_FAISS_LIBRARY
[Accessed 22 January 2026].
- Gao, Y. et al., 2024. *Retrieval-Augmented Generation for Large Language Models: A Survey*. [Online]
Available at: <https://arxiv.org/pdf/2312.10997>
[Accessed 12 August 2025].
- Goyal, N. et al., 2020. *Unsupervised Cross-lingual Representation Learning at Scale*. [Online]
Available at: <https://aclanthology.org/2020.acl-main.747.pdf>
[Accessed 11 August 2025].
- Huang, L. et al., 2023. *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*. [Online]
Available at: <https://arxiv.org/abs/2311.05232>
[Accessed 29 July 2025].
- Johnson, S. & Hyland-Wood, D., 2024. [Online]
Available at: <https://arxiv.org/html/2412.04503v1#S4>
[Accessed 12 August 2025].
- Karpukhin, V. et al., 2020. *Dense Passage Retrieval for Open-Domain Question Answering*. [Online]
Available at: <https://aclanthology.org/2020.emnlp-main.550.pdf>
[Accessed 9 August 2025].
- Kelk, I., 2023. *What Is RAG? (Retrieval Augmented Generation)*. [Online]
Available at: <https://www.clarifai.com/blog/what-is-rag-retrieval-augmented-generation>
[Accessed 11 May 2025].
- Kshirsagar, A., 2024. *Enhancing RAG Performance Through Chunking and Text Splitting Techniques*. [Online]
Available at: https://www.researchgate.net/publication/383998204_Enhancing_RAG_Performance_Through_Chunking_and_Text_Splitting_Techniques
[Accessed 12 August 2025].
- LeCun, Y., Bengio, Y. & Hinton, G., 2015. *Deep Learning*. [Online]
Available at: https://www.researchgate.net/publication/277411157_Deep_Learning
[Accessed 8 August 2025].

- Lewis , P., Perez , E., Piktus, A. & Petroni , F., 2020. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. [Online]
Available at: https://www.researchgate.net/publication/341639856_Retrieval-Augmented_Generation_for_Knowledge-Intensive_NLP_Tasks
[Accessed 28 July 2025].
- Machidon, O., 2025. *Comparing OCR Pipelines for Folkloristic Text Digitization*. [Online]
Available at: <https://arxiv.org/html/2507.19092v1>
[Accessed 11 August 2025].
- Panda, S. P., Behera, V., Pradhan, A. & Mohanty, A., 2019. *A Rule-based Information Extraction System*. [Online]
Available at: https://www.researchgate.net/publication/364660732_A_Rule-based_Information_Extraction_System
[Accessed 7 August 2025].
- Pande, V. & Khandelwal, A. S., 2016. *IOSR Journal of Computer Engineering*. [Online]
Available at: <https://www.iosrjournals.org/iosr-jce/papers/conf.15013/Volume%209/4.%2016-20.pdf>
[Accessed 7 August 2025].
- Reimers, N. & Gurevych, I., 2019. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. [Online]
Available at: <https://arxiv.org/abs/1908.10084>
[Accessed 9 August 2025].
- Thakur, N. et al., 2021. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. [Online]
Available at: <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/file/65b9eea6e1cc6bb9f0cd2a47751a186f-Paper-round2.pdf>
[Accessed 12 August 2025].
- Topsakal, O. & Akinci, T. C., 2023. *Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast*. [Online]
Available at:
https://www.researchgate.net/publication/372669736_Creating_Large_Language_Model_Applications_Utilizing_LangChain_A_Primer_on_Developing_LLM_Apps_Fast
[Accessed 10 August 2025].
- Wu, S. et al., 2023. *BloombergGPT: A Large Language Model for Finance*. [Online]
Available at: <https://arxiv.org/pdf/2303.17564>
[Accessed 13 August 2025].
- Xu, Y. et al., 2021. *LayoutLMv2: Multi-modal Pre-training for Visually-rich Document Understanding*. [Online]
Available at: <https://aclanthology.org/2021.acl-long.201.pdf>
[Accessed 17 January 2026].
- Zhang, Q. et al., 2024. *Document Parsing Unveiled: Techniques, Challenges, and Prospects for Structured Information Extraction*. [Online]
Available at: <https://arxiv.org/html/2410.21169v1>
[Accessed 7 August 2025].
- Zhang, Y., Zhou, Q., Imani, F. & Tang, J., 2025. *Seeing the Unseen: Towards Zero-Shot Inspection for Wind Turbine Blades using Knowledge-Augmented Vision Language Models*. [Online]

Available at: <https://arxiv.org/pdf/2510.22868>
[Accessed 20 January 2026].

Appendices

financial_documents_processing.py:

```
# Here I am importing all the necessary libraries needed for the entire process of RAG.
# FAISS handles vector similarity search, while the OpenAI and LangChain libraries are
# used to work with Azure OpenAI models. Also, environment variables are loaded from a
# .env file.

from openai import AzureOpenAI
from langchain_openai import AzureChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_openai import AzureOpenAIEmbeddings
from langchain.chains.question_answering import load_qa_chain
from langchain.output_parsers import ResponseSchema, StructuredOutputParser

from doc_text_extraction import extract_text_from_file_markdown
import os
import dotenv
import time
from datetime import datetime

import faiss
import numpy as np

dotenv.load_dotenv(dotenv.find_dotenv())

AZURE_ENDPOINT = os.environ.get("AZURE_ENDPOINT")
OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY")
MODEL_GPT_4o = os.environ.get("MODEL_GPT_4o")
MODEL_GPT_4o_MINI = os.environ.get("MODEL_GPT_4o_MINI")
API_VERSION = os.environ.get("API_VERSION")
TEXT_EMBEDDING = os.environ.get("TEXT_EMBEDDING")

# This class is in charge of one financial document and all of the steps that go into processing it.
# It extracts documents, makes embeddings, builds an FAISS index, and allows you to run queries based
# on retrieval. When the document is made, it is processed and ready for semantic search. The PDF is
# turned into structured Markdown content, and each page gets an embedding. The vectors are then stored
# in an FAISS index. The GPT model is accessed through Azure OpenAI, which is used later for question-
# answering stage. Then, the embedding model converts text into numerical vector which represent semantic
# meaning. The PDF file is then processed and converted into structural markdown representations of the page
```

```
# so that it keeps the alignment of the table. After, each page of text is transformed into a vector
# representation so that semantic similarity searches can later be performed. Lastly, a FAISS index is
# created to store all the page embeddings to allow efficient similarity search.
```

```
class FincialDocument:
```

```
    def __init__(
        self,
        path: str,
        temp_folder: str,
        openai_key: str = OPENAI_API_KEY,
        verbose: bool = True,
    ):

        # Verbosity and logging
        self.verbose = verbose
        if self.verbose:
            print(f'{datetime.now().strftime("%H:%M:%S")} | Initialising document...')

        # OpenAI Model
        self.model = AzureChatOpenAI(
            openai_api_key=OPENAI_API_KEY,
            azure_endpoint=AZURE_ENDPOINT,
            deployment_name=MODEL_GPT_4o_MINI,
            openai_api_version=API_VERSION,
            openai_api_type="azure",
            temperature=0,
        )

        # OpenAI Embeddings
        self.embeddings = AzureOpenAIEmbeddings(
            azure_deployment=TEXT_EMBEDDING,
            model="text-embedding-ada-002",
            openai_api_version=API_VERSION,
            openai_api_key=OPENAI_API_KEY,
            openai_api_type="azure",
            azure_endpoint=AZURE_ENDPOINT,
        )

        self.client = AzureOpenAI(
            azure_endpoint=AZURE_ENDPOINT,
            api_key=OPENAI_API_KEY,
```

```

        api_version=API_VERSION,
    )

    self.path = path
    self.openai_key = openai_key

    texts_extracted = extract_text_from_file_markdown(
        self.path, openai_key, workers=15, temp_folder=temp_folder
    )

    self.texts_search = sorted(
        texts_extracted, key=lambda x: int(x.metadata["page_num"])
    )

    texts_to_embed = [item.page_content for item in self.texts_search]

    embeddings = self.embeddings.embed_documents(texts_to_embed)

    embedding_dim = len(embeddings[0])
    embedding_matrix = np.array(embeddings, dtype='float32')

    self.index = faiss.IndexFlatL2(embedding_dim)

    self.index.add(embedding_matrix)

# Converts the user query into a vector representation by applying the same embedding model to the content
# pages.
def embed_query(self, query: str):
    """Embed a query string using the same embedding client."""
    return np.array(self.embeddings.embed_query(query), dtype='float32')

# Finds the most relevant document by comparing embedded query vector with the stored vectors in the FAISS
# index
def retrieve_top_k_documents(self, query: str, k: int = 5):
    """Retrieve top k documents most similar to the query."""
    query_embedding = self.embed_query(query).reshape(1, -1)
    distances, indices = self.index.search(query_embedding, k)
    top_docs = [self.texts_search[i] for i in indices[0]]
    return top_docs

```

```

# This step retrieves relevant document pages and asks the language model to find the company's reported revenue.
# The output must follow a structured schema.

def revenue(self):

    schema = ResponseSchema(
        name="revenue",
        description="The value of the revenue",
    )

    output_parser = StructuredOutputParser.from_response_schemas([schema])

    format_instructions = output_parser.get_format_instructions()

    template = """

    You are a financial analyst assistant specialised in interpreting consolidated financial statements of companies.

    Given the following extracted document content, please provide clear and concise answers to the question below. Focus only on information explicitly stated in the document.

    Total revenue can also be referred to as sales.

    If there is no information around revenue return "N/A" as answer.

    {format_instructions}

    Question: {query}
    Answer:
    """

    prompt_template = PromptTemplate(
        template=template,
        partial_variables={"format_instructions": format_instructions},
        input_variables=["query"],
    )

    query = """What is the total revenue?"""

```

```

prompt = prompt_template.format(query=query)

top_docs = self.retrieve_top_k_documents(query, k=5)

chain = load_qa_chain(
    llm=self.model,
    chain_type="stuff",
)

attempts = 0
while attempts < 10:
    try:
        output = chain(
            {
                "input_documents": top_docs,
                "question": prompt,
                "query": query,
            },
            return_only_outputs=True,
        )
        output = output["output_text"]

        response_as_dict = output_parser.parse(output)
        break
    except Exception as e:
        attempts += 1
        print(f"Attempt {attempts} failed with error: {str(e)}")
        time.sleep(2)
    else:
        raise Exception(f"Code failed after {10} attempts")

return response_as_dict["revenue"]

def profit(self):

    schema = ResponseSchema(
        name="profit",
        description="The value of the profit",
    )

```

```

output_parser = StructuredOutputParser.from_response_schemas([schema])

format_instructions = output_parser.get_format_instructions()

template = """

You are a financial analyst assistant specialised in interpreting consolidated financial statements of
companies.

Given the following extracted document content, please provide clear and concise answers to the question
below. Focus only on information explicitly stated in the document.

Profit can also be referred to as net income.

If there is no information around profit or net income return "N/A" as answer.

{format_instructions}

Question: {query}
Answer:
"""

prompt_template = PromptTemplate(
    template=template,
    partial_variables={"format_instructions": format_instructions},
    input_variables=["query"],
)

query = """What is the profit for the reporting period?"""

prompt = prompt_template.format(query=query)

top_docs = self.retrieve_top_k_documents(query, k=5)

chain = load_qa_chain(
    llm=self.model,
    chain_type="stuff",
)

attempts = 0

```

```

while attempts < 10:
    try:
        output = chain(
            {
                "input_documents": top_docs,
                "question": prompt,
                "query": query,
            },
            return_only_outputs=True,
        )
        output = output["output_text"]

        response_as_dict = output_parser.parse(output)
        break
    except Exception as e:
        attempts += 1
        print(f"Attempt {attempts} failed with error: {str(e)}")
        time.sleep(2)
    else:
        raise Exception(f"Code failed after {10} attempts")

    return response_as_dict["profit"]

def assets(self):

    schema = ResponseSchema(
        name="assets",
        description="The value of the total assest",
    )

    output_parser = StructuredOutputParser.from_response_schemas([schema])

    format_instructions = output_parser.get_format_instructions()

    template = """

    You are a financial analyst assistant specialised in interpreting consolidated financial statements of
    companies.

```

Given the following extracted document content, please provide clear and concise answers to the question below. Focus only on information explicitly stated in the document.

If there is no information around total assets return "N/A" as answer.

{format_instructions}

Question: {query}

Answer:

"""

```
prompt_template = PromptTemplate(
    template=template,
    partial_variables={"format_instructions": format_instructions},
    input_variables=["query"],
)

query = """What are the total assets as mentioned in the document?"""

prompt = prompt_template.format(query=query)

top_docs = self.retrieve_top_k_documents(query, k=5)

chain = load_qa_chain(
    llm=self.model,
    chain_type="stuff",
)

attempts = 0
while attempts < 10:
    try:
        output = chain(
            {
                "input_documents": top_docs,
                "question": prompt,
                "query": query,
            },
            return_only_outputs=True,
        )
        output = output["output_text"]
```

```

        response_as_dict = output_parser.parse(output)
        break
    except Exception as e:
        attempts += 1
        print(f"Attempt {attempts} failed with error: {str(e)}")
        time.sleep(2)
    else:
        raise Exception(f"Code failed after {10} attempts")

    return response_as_dict["assets"]

def equity(self):

    schema = ResponseSchema(
        name="equity",
        description="The value of the shareholders' equity",
    )

    output_parser = StructuredOutputParser.from_response_schemas([schema])

    format_instructions = output_parser.get_format_instructions()

    template = """

    You are a financial analyst assistant specialised in interpreting consolidated financial statements of
    companies.

    Given the following extracted document content, please provide clear and concise answers to the question
    below. Focus only on information explicitly stated in the document.

    If there is no information around shareholders' equity return "N/A" as answer.

    {format_instructions}

    Question: {query}
    Answer:
    """

    prompt_template = PromptTemplate(

```

```

        template=template,
        partial_variables={"format_instructions": format_instructions},
        input_variables=["query"],
    )

    query = """What is the shareholders' equity reported?"""

    prompt = prompt_template.format(query=query)

    top_docs = self.retrieve_top_k_documents(query, k=5)

    chain = load_qa_chain(
        llm=self.model,
        chain_type="stuff",
    )

    attempts = 0
    while attempts < 10:
        try:
            output = chain(
                {
                    "input_documents": top_docs,
                    "question": prompt,
                    "query": query,
                },
                return_only_outputs=True,
            )
            output = output["output_text"]

            response_as_dict = output_parser.parse(output)
            break
        except Exception as e:
            attempts += 1
            print(f"Attempt {attempts} failed with error: {str(e)}")
            time.sleep(2)
    else:
        raise Exception(f"Code failed after {10} attempts")

    return response_as_dict["equity"]

```

```

def earnings_per_share(self):

    schema = ResponseSchema(
        name="earnings_per_share",
        description="The value of the earnings per share",
    )

    output_parser = StructuredOutputParser.from_response_schemas([schema])

    format_instructions = output_parser.get_format_instructions()

    template = """

    You are a financial analyst assistant specialised in interpreting consolidated financial statements of
    companies.

    Given the following extracted document content, please provide clear and concise answers to the question
    below. Focus only on information explicitly stated in the document.

    If there is no information around earnings per share return "N/A" as answer.

    {format_instructions}

    Question: {query}
    Answer:
    """

    prompt_template = PromptTemplate(
        template=template,
        partial_variables={"format_instructions": format_instructions},
        input_variables=["query"],
    )

    query = """What is the earnings per share (EPS)?"""

    prompt = prompt_template.format(query=query)

    top_docs = self.retrieve_top_k_documents(query, k=5)

    chain = load_qa_chain(

```

```

        llm=self.model,
        chain_type="stuff",
    )

    attempts = 0
    while attempts < 10:
        try:
            output = chain(
                {
                    "input_documents": top_docs,
                    "question": prompt,
                    "query": query,
                },
                return_only_outputs=True,
            )
            output = output["output_text"]

            response_as_dict = output_parser.parse(output)
            break
        except Exception as e:
            attempts += 1
            print(f"Attempt {attempts} failed with error: {str(e)}")
            time.sleep(2)
    else:
        raise Exception(f"Code failed after {10} attempts")

    return response_as_dict["earnings_per_share"]

def cash_flow(self):

    schema = ResponseSchema(
        name="cash_flow",
        description="The value of the cash flows",
    )

    output_parser = StructuredOutputParser.from_response_schemas([schema])

    format_instructions = output_parser.get_format_instructions()

    template = """

```

You are a financial analyst assistant specialised in interpreting consolidated financial statements of companies.

Given the following extracted document content, please provide clear and concise answers to the question below. Focus only on information explicitly stated in the document.

cash flows can be values from from operating, investing, and financing activities.

If there is no informarion around cash flows return "N/A" as answer.

{format_instructions}

Question: {query}

Answer:

"""

```
prompt_template = PromptTemplate(
    template=template,
    partial_variables={"format_instructions": format_instructions},
    input_variables=["query"],
)
```

```
query = """What are the cash flows value?"""
```

```
prompt = prompt_template.format(query=query)
```

```
top_docs = self.retrieve_top_k_documents(query, k=5)
```

```
chain = load_qa_chain(
    llm=self.model,
    chain_type="stuff",
)
```

```
attempts = 0
```

```
while attempts < 10:
```

```
    try:
```

```
        output = chain(
```

```
            {
```

```
                "input_documents": top_docs,
```

```

        "question": prompt,
        "query": query,
    },
    return_only_outputs=True,
)
output = output["output_text"]

response_as_dict = output_parser.parse(output)
break
except Exception as e:
    attempts += 1
    print(f"Attempt {attempts} failed with error: {str(e)}")
    time.sleep(2)
else:
    raise Exception(f"Code failed after {10} attempts")

return response_as_dict["cash_flow"]

def contingent(self):

    schema = ResponseSchema(
        name="contingent",
        description="The value of contingent liabilities",
    )

    output_parser = StructuredOutputParser.from_response_schemas([schema])

    format_instructions = output_parser.get_format_instructions()

    template = """

    You are a financial analyst assistant specialised in interpreting consolidated financial statements of
companies.

    Given the following extracted document content, please provide clear and concise answers to the question
below. Focus only on information explicitly stated in the document.

    If there is no information around contingent liabilities return "N/A" as answer.

    {format_instructions}

```

```

Question: {query}

Answer:
"""

prompt_template = PromptTemplate(
    template=template,
    partial_variables={"format_instructions": format_instructions},
    input_variables=["query"],
)

query = """What contingent liabilities are mentioned?"""

prompt = prompt_template.format(query=query)

top_docs = self.retrieve_top_k_documents(query, k=5)

chain = load_qa_chain(
    llm=self.model,
    chain_type="stuff",
)

attempts = 0
while attempts < 10:
    try:
        output = chain(
            {
                "input_documents": top_docs,
                "question": prompt,
                "query": query,
            },
            return_only_outputs=True,
        )
        output = output["output_text"]

        response_as_dict = output_parser.parse(output)
        break
    except Exception as e:
        attempts += 1
        print(f"Attempt {attempts} failed with error: {str(e)}")

```

```

        time.sleep(2)
    else:
        raise Exception(f"Code failed after {10} attempts")

    return response_as_dict["contingent"]

# This function runs all individual metric extraction functions and returns the results in a single structure.
def extract_data(self):
    return {"revenue": self.revenue(),
            "profit": self.profit(),
            "assets_and_liabilities": self.assets(),
            "equity": self.equity(),
            "earnings_per_share": self.earnings_per_share(),
            "cash_flow": self.cash_flow(),
            "contingent": self.contingent(),
            }

```

doc text extraction.py:

```

# This section of the code handles page by page extraction from PDFs by converting PDFs into images,
# sending each image to the azure api and then returns the extracted content as Langchain document
# information
import base64
import concurrent
from concurrent.futures import ThreadPoolExecutor
import glob
import os
import re
from threading import current_thread
import time

from openai import AzureOpenAI, InternalServerError, APIStatusError
from langchain.schema import Document

import utils
import dotenv

dotenv.load_dotenv(dotenv.find_dotenv())

# Loading the azure configuration
AZURE_ENDPOINT = os.environ.get("AZURE_ENDPOINT")

```

```

OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY")
MODEL_GPT_4o = os.environ.get("MODEL_GPT_4o")
MODEL_GPT_4o_MINI = os.environ.get("MODEL_GPT_4o_MINI")
API_VERSION = os.environ.get("API_VERSION")

# Creating a repeated AzureOpenAI client, in order to call for the chat completions endpoint.
# To minimise shared-state problems during parallel processing, a separate client is built for
# each worker thread.
def init_openai_client(API_KEY: str):
    return AzureOpenAI(
        azure_endpoint=AZURE_ENDPOINT,
        api_key=OPENAI_API_KEY,
        api_version=API_VERSION,
    )

# A single page image's structured content is extracted, and Markdown formatting is used to try
# to maintain tables. To prevent crashes while running in parallel, each worker thread chooses
# its own OpenAI client. The function includes retry logic where it starts with a smaller model
# for cost, it falls back to the larger model on the final try if needed, and detects a known
# failure mode where the output becomes repetitive.
def extract_text_from_page_markdown(
    openai_clients, page_path: str, role_msg: str, instruction_msg: str
):
    thread = current_thread()
    thread_number = int(thread.name.split("_")[-1])
    openai_client = openai_clients[thread_number]

    page_num = page_path.split("_p")[-1].split(".jpg")[0]
    file_basename = os.path.basename(page_path)

    b64_img = utils.encode_image(page_path)
    model_name = MODEL_GPT_4o_MINI
    attempts = 5
    while attempts > 0:
        try:
            response = openai_client.chat.completions.create(
                model=model_name,
                messages=[
                    {"role": "system", "content": role_msg},
                    {

```

```

        "role": "user",
        "content": [
            {"type": "text", "text": instruction_msg},
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/png;base64,{b64_img}"
                },
            },
        ],
    },
],
temperature=0.0,
)
output = response.choices[0].message.content

if (
    attempts > 1
    and detect_pattern_of_dots(output)
    or output
    == "I'm unable to extract text from the image directly. However, if you can provide the text or details
from the image, I can help you analyze or summarize it."
):
    print(
        f"The data was not extracted correctly from the page, attempts left: {attempts}"
    )
    raise ValueError(
        f"The data was not extracted correctly from the page, attempts left: {attempts}"
    )

output = remove_output_sys_messages(output)

doc = Document(
    page_content=output,
    metadata={
        "source": file_basename,
        "page_num": page_num,
        "usage": response.usage,
    },
)

```

```

os.remove(page_path)

return doc

except:

    attempts -= 1

    if attempts == 1:

        model_name = MODEL_GPT_4o

    else:

        return None

# Same Markdown extractor but needed when extraction behaviour is switched later.
def extract_text_from_page(
    openai_clients, page_path: str, role_msg: str, instruction_msg: str
):

    thread = current_thread()
    thread_number = int(thread.name.split("_")[-1])
    openai_client = openai_clients[thread_number]

    page_num = page_path.split("_p")[-1].split(".jpg")[0]
    file_basename = os.path.basename(page_path)

    b64_img = utils.encode_image(page_path)
    model_name = MODEL_GPT_4o_MINI
    attempts = 5
    while attempts > 0:

        try:

            response = openai_client.chat.completions.create(
                model=model_name,
                messages=[
                    {"role": "system", "content": role_msg},
                    {
                        "role": "user",
                        "content": [
                            {"type": "text", "text": instruction_msg},
                            {
                                "type": "image_url",
                                "image_url": {
                                    "url": f"data:image/png;base64,{b64_img}"
                                }
                            },
                        ],
                    },
                ],
            )

```

```

        ],
    },
],
temperature=0.0,
)

output = response.choices[0].message.content

if (
    attempts > 1
    and detect_pattern_of_dots(output)
    or output
    == "I'm unable to extract text from the image directly. However, if you can provide the text or details
from the image, I can help you analyze or summarize it."
):
    print(
        f"The data was not extracted correctly from the page, attempts left: {attempts}"
    )
    raise ValueError(
        f"The data was not extracted correctly from the page, attempts left: {attempts}"
    )

output = remove_output_sys_messages(output)

doc = Document(
    page_content=output,
    metadata={
        "source": file_basename,
        "page_num": page_num,
        "usage": response.usage,
    },
)
os.remove(page_path)
return doc

except:
    attempts -= 1
    if attempts == 1:
        model_name = MODEL_GPT_4o
    else:

```

```

return None

# Splitting a list into chunks.
def chunk_list(lst, chunk_size):
    for i in range(0, len(lst), chunk_size):
        yield lst[i : i + chunk_size]

# Defining the parallel text extraction which runs the page across multiple threads, so long
# documents can be processed quickly. This creates one Azure client per worker, submits
# extraction jobs to a thread pool, and collects the results and returns them as a
# document.
def parallel_text_extraction_markdown(API_KEY: str, img_paths: list[str], workers: int):

    role_msg = """
        You are a professional image-to-text document parser.
        Extract all text from these images, treating them as pages of a PDF document.
        Try to format any tables found in the images.
        Do not include page numbers, page headers, or page footers.
        Return only the extracted text. No commentary.
        **Exclude Nothing**: The only text that does not need to be included is the page number.
        Return the text as a Markdown document.
        1. Keep the source language.
        2. Include headers and footers.
        3. Don't interpolate or make up data.
        4. Extract all text and tables as Markdown
        5. Here's a comprehensive list of Markdown formatting guidelines that cover various aspects of text
        formatting, lists, tables, links, images, and more:

        1. Headings: Use `#` for headings. The number of `#` symbols indicates the heading level
        (1-6).
        - Example:
        - `# Heading 1`
        - `## Heading 2`
        - `### Heading 3`

        2. Bold Text: Use double asterisks `**` or double underscores `__` to make text bold.
        - Example: bold text or bold text

        3. Italic Text: Use single asterisks `*` or single underscores `_` to italicize text.
        - Example: italic text or italic text
    """

```

4. ****Strikethrough****: Use double tildes `~~` to strikethrough text.

- Example: `~~strikethrough~~`

5. ****Blockquotes****: Use the `>` character to create blockquotes.

- Example:

```
'''
> This is a blockquote.
'''
```

6. ****Unordered Lists****: Use asterisks `\*`, plus `+`, or hyphens `-` to create unordered lists.

- Example:

```
'''
- Item 1
- Item 2
  - Subitem 2.1
'''
```

7. ****Ordered Lists****: Use numbers followed by a period to create ordered lists.

- Example:

```
'''
1. First item
2. Second item
  1. Subitem 2.1
'''
```

8. ****Links****: Use square brackets for the link text and parentheses for the URL.

- Example: `[Link text](http://example.com)`

9. ****Images****: Use an exclamation mark `!` followed by square brackets for the alt text and parentheses for the image URL.

- Example: `![Alt text](http://example.com/image.jpg)`

10. ****Horizontal Rules****: Use three or more hyphens `---`, asterisks `\*\*\*`, or underscores `\_\_\_` to create horizontal rules.

- Example:

```
'''
---
'''
```

11. **Line Breaks**: End a line with two or more spaces and press Enter, or use `<br>` for a line break.

12. **Footnotes**: Use square brackets with a caret `[^1]` for footnotes, and define them at the bottom.

- Example:

```
...
```

This is a sentence with a footnote.^[1]

^[1]: This is the footnote text.

```
...
```

13. **Escaping Characters**: Use a backslash to escape special Markdown characters.

- Example: ``not italic``

14. **Task Lists**: Use `- [ ]` for incomplete tasks and `- [x]` for completed tasks.

- Example:

```
...
```

- [] Task 1

- [x] Task 2

```
...
```

15. **Nested Lists**: Indent sub-items with two spaces or a tab.

- Example:

```
...
```

- Item 1

- Subitem 1.1

```
...
```

6. For tables, Markdown require a specific format to be rendered correctly:

1. Header Separator Consistency: Ensure that the header separator row contains the same number of columns as the header row. Use hyphens (-) to create the separator.

2. Column Alignment: Maintain proper alignment of columns in both the header and data rows to enhance readability, even if explicit alignment is not required by Markdown.

3. Uniform Column Count: Verify that each data row has the same number of cells as the header row to prevent misalignment and confusion.

4. Cell Spacing: Include a space before and after the content within each cell to improve readability.

5. Consistent Data Formatting: Use a uniform format for similar types of data (e.g., currency, percentages) throughout the table to ensure clarity.

6. Header Repetition: If the table extends beyond a single page, ensure that the table header is repeated on each page for easier reference.

7. Avoid Extra Columns: Check for and eliminate any extra columns in the header or data rows that do not align with the intended structure of the table.

7. Make sure to include all amounts in euros or dollars, percentages (%), addresses, dates, numbers, and tables.

```
"""

instruction_msg = (
    "Extract all text, tables, footnotes, and citations from the image"
)

MAX_NUM_WORKERS = 15

if workers > MAX_NUM_WORKERS:
    raise ValueError(
        f"Max number of workers exceeded. The max number of workers is {MAX_NUM_WORKERS}; this prevents reaching the rate limits"
    )

workers = min(workers, len(img_paths))

openai_clients_list = [init_openai_client(API_KEY) for _ in range(workers)]
processes = []
with ThreadPoolExecutor(max_workers=workers) as executor:
    for page in img_paths:
        input_data = {
            "openai_clients": openai_clients_list,
            "page_path": page,
            "role_msg": role_msg,
            "instruction_msg": instruction_msg,
        }
        processes.append(
            executor.submit(extract_text_from_page_markdown, **input_data)
        )

results = concurrent.futures.as_completed(processes)
```

```

    results = [r.result() for r in results]
    return results

# Converts a PDF into page images, transfers them through the parallel extraction system,
# and returns a list of page-level documents.
def extract_text_from_file_markdown(
    file_path: str, API_KEY: str, workers: int = 1, temp_folder: str = None
) -> list[Document]:

    file_basename = utils.pdf_to_image(
        file_path, temp_folder=temp_folder
    )
    img_paths = glob.glob(f"{temp_folder}/tmp_*.")
    results = parallel_text_extraction_markdown(API_KEY, img_paths, workers)

    return results

# This function flags the failed extractions that return long repetitive sequences so that
# the page can be retried.
def detect_pattern_of_dots(text):
    pattern = r"(. . .)"
    pattern = rf"({' . . . ' * 30})"
    matches = re.findall(pattern, text)
    if len(matches) > 1:
        print(f"Dots detected, the pattern repeats {len(matches)} times")
        return True
    else:
        return False

# The system here is removing unnecessary text that is retrieved with the data so that
# only the data is extracted.
def remove_output_sys_messages(text):
    sys_messages = [
        "### Transcription of the Document",
        "If you need further assistance or details, feel free to ask!",
        "Here is the transcription of the text from the document:",
        "Here is the text extracted from the image:",
        "Let me know if you need any further assistance!",
        "There are no diagrams, tables, plots, or charts present in the image.",
    ]

```

```

    "This text includes all relevant details, including names, numbers, and descriptions from the document. Here
is the extracted text from the image:",

    "There are no diagrams, tables, plots, or charts in the image. Here is the extracted text from the image:",

    "This text includes all the relevant details from the document. Here is the extracted text from the image:",

    "This text includes all the relevant information from the document, including the details of the insured
properties, coverage limits, and deductibles. Here is the extracted text from the image:",

    "This text includes all the relevant information from the document. Here is the extracted text from the
image:",

    "No diagrams, tables, plots, or charts are present in the image.",

    "This text includes all relevant details from the document. If you need further assistance, feel free to ask!
Here is the extracted text from the image:",

    "Here is the extracted text from the image:",

    "This text includes all the relevant details from the document. If you need further assistance, feel free to
ask!",

    "This text includes all the relevant information from the document.",
    "",
    "If you need further assistance, feel free to ask!",
    "",
    "This text includes all relevant details, including names, amounts, and descriptions from the document.",
    "",
    "***Referencia a 'ajuar':** No se encontró ninguna referencia a 'ajuar' en el texto extraído.",
    "No references to 'ajuar' were found in the text.",
    "This text includes all relevant information, including references to \"ajuar\" and any amounts or percentages
found in the original document.',
    'No specific references to \"ajuar\", \"prima\", \"limites\", or \"sumas aseguradas\" were found in the text.',
    \"```markdown\",
    \"```\",

    "I'm unable to extract text from the image directly. However, if you can provide the text or details from the
document, I can help you format it or analyze it as needed.",

    "No references to 'ajuar', 'prima', 'limites', 'sumas aseguradas', or 'tomador del seguro' were found.",
    "This text includes all relevant information as requested.",
    "Extracted Text",
    "Here is the extracted text from the image, formatted in Markdown:",
    "markdown",

    "I'm unable to extract text from images directly. If you can provide the text or details from the image, I'd be
happy to help format it or assist you with any specific requests!",

]

for message in sys_messages:
    text = text.replace(message, "")

return text

```

utils.py:

```
# Importing libraries
import base64
import os
import fitz

# This function opens a PDF document and converts every page into a high-resolution image.
# Each document is saved as a JPG file temporarily. These images are then used for vision-
# based text extraction later on.
def pdf_to_image(path: str, temp_folder: str):
    basename = os.path.basename(path)
    doc = fitz.open(path)
    mat = fitz.Matrix(2, 2)

    # num_pages_to_process = min(2, doc.page_count)
    for page in range(doc.page_count):
        fitz_page = doc.load_page(page)
        save_path = f"{temp_folder}/tmp_{basename}_p{page+1}.jpg"
        pixmap = fitz_page.get_pixmap(matrix=mat)
        pixmap.save(save_path)
    return basename

# This function reads an image file and converts it into a base64 encoded string.
# This format is required when sending images to the OpenAI API for multimodal processing.
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        base64_image = base64.b64encode(image_file.read()).decode("utf-8")
    return base64_image
```

run.ipynb

```
import os
from financial_documents_processing import FincialDocument

import warnings

warnings.filterwarnings("ignore")

# Here I am writing the directory for the datasets file.
fincial_data_folder_path = "Datasets"
```

```

# This function below extracts all the data asked by the queries accordingly,
# and then the data is put into an excel file using pandas to create a table.
extracted_data_dict = {}
paths = os.listdir(fincial_data_folder_path)

for path in paths:
    if path.endswith(".pdf"):
        file_path = os.path.join(fincial_data_folder_path, path)
        extracted_data = FincialDocument(file_path, temp_folder= "temp_folder")
        data = extracted_data.extract_data()
        extracted_data_dict[path] = data

import pandas as pd

pd.DataFrame(extracted_data_dict).to_excel("results.xlsx")

```

evaluation.py:

```

# Importing the libraries
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
)

# Loading the excel files produced by the extraction system.
def load_results(excel_path: str) -> pd.DataFrame:
    if not os.path.exists(excel_path):
        raise FileNotFoundError(f"Could not find: {excel_path}")
    return pd.read_excel(excel_path)

# Converting individual cells in the excel files into '0' for missing values and '1'

```

```

# for anything else.
def cell_to_label(x) -> int:
    if pd.isna(x):
        return 0

    s = str(x).strip().lower()

    if s in {"not available", "n/a", ""}:
        return 0

    return 1

# Converting the results table into a single list of values so it can be evaluated.
def to_binary_availability(df: pd.DataFrame, metric_col_hint: str = "Unnamed: 0") -> np.ndarray:

    data = df.copy()

    if metric_col_hint in data.columns:
        data = data.drop(columns=[metric_col_hint])

    bin_df = data.applymap(cell_to_label)

    return bin_df.to_numpy().ravel()

# Creating a confusion matrix to show the true positives and false negatives of the system.
def plot_confusion_matrix(y_true: np.ndarray, y_pred: np.ndarray, out_path: str) -> None:
    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])

    fig, ax = plt.subplots()
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
    disp.plot(ax=ax, colorbar=True)
    ax.set_title("Confusion Matrix (Availability-Based)")
    fig.tight_layout()
    fig.savefig(out_path, dpi=300)
    plt.close(fig)

# Creating an overall metrics performance chart.
def plot_overall_metrics(y_true: np.ndarray, y_pred: np.ndarray, out_path: str) -> dict:
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)

```

```

rec = recall_score(y_true, y_pred, zero_division=0)
f1 = f1_score(y_true, y_pred, zero_division=0)

metrics = {"Accuracy": acc, "Precision": prec, "Recall": rec, "F1-Score": f1}

fig, ax = plt.subplots()
ax.bar(list(metrics.keys()), list(metrics.values()))
ax.set_ylim(0, 1)
ax.set_title("Overall Performance Metrics (Availability-Based)")
fig.tight_layout()
fig.savefig(out_path, dpi=300)
plt.close(fig)

return metrics

# Creating a correct vs incorrect chart.
def plot_correct_incorrect(y_true: np.ndarray, y_pred: np.ndarray, out_path: str) -> tuple[int, int]:
    correct = int(np.sum(y_pred == y_true))
    incorrect = int(np.sum(y_pred != y_true))

    fig, ax = plt.subplots()
    ax.bar(["Correct", "Incorrect"], [correct, incorrect])
    ax.set_title("Correct vs Incorrect (Availability Labels)")
    fig.tight_layout()
    fig.savefig(out_path, dpi=300)
    plt.close(fig)

    return correct, incorrect

# Here, I am running the whole evaluation from end to end by loading excel, converting outputs to availability
# labels, assuming all existing metrics are correct (1) and saves all the graphs created.
def main() -> None:
    excel_path = "results.xlsx"

    df = load_results(excel_path)

    y_pred = to_binary_availability(df, metric_col_hint="Unnamed: 0")

    y_true = np.ones_like(y_pred)

```

```

plot_confusion_matrix(y_true, y_pred, "confusion_matrix.png")
metrics = plot_overall_metrics(y_true, y_pred, "overall_metrics.png")
correct, incorrect = plot_correct_incorrect(y_true, y_pred, "correct_vs_incorrect.png")

print("Availability-Based Evaluation Summary")
print(f"Total outputs evaluated: {len(y_pred)}")
print(f"Correct (matches expected availability): {correct}")
print(f"Incorrect (does not match expected availability): {incorrect}")
print("Metrics:")
for k, v in metrics.items():
    print(f"  {k}: {v:.4f}")
print("Saved figures:")
print("confusion_matrix.png")
print("overall_metrics.png")
print("correct_vs_incorrect.png")

if __name__ == "__main__":
    main()

```